Spectral Methods in Julia

March 4, 2026

Contents

0.1 Chapter 1

0.1.1 Program 1

```
using CairoMakie, LaTeXStrings
using ToeplitzMatrices
"p1 - convergence of fourth -order finite differences"
function p1()
    # For various N, set up grid in [ - \pi, \pi] and function u(x):
    N = [2^j for j in 3:12]
    errors = []
    for N in N
        h = 2 \pi / N
        x = [ - \pi + j * h for j in 1:N]
        u = @. exp(sin(x)^2)
        uprime = @. 2sin(x) * cos(x) * u

        # Construct sparse fourth -order differentiation matrix:
        col = zeros(N)
        col[[2, 3, N -1, N]] = [ -2/3, 1/12, -1/12, 2/3] / h
        D = Toeplitz(col, -col)

        # Compute max of abs(D*u - uprime):
        error = maximum(abs, D * u - uprime)
        push!(errors, error)
    end

    fig = Figure()
    ax = Axis(fig[1, 1]; title="Convergence of fourth -order finite differences",
        xscale=log10, yscale=log10, xlabel=L"N", ylabel="error")
    scatter!(ax, N, errors)
    lines!(ax, N, 1 ./ N .^ 4; linestyle=:dash, color=:gray)
    text!(ax, 105, 1e -8; text=L"N^{ -4}")
    return fig
end
```

Output 1

```
using SpectralMethodsTrefethen
p1()
```

There is so much to cover as we get started!

First, notice that I've presented Program 1 as a function. This is the most natural way to do things in Julia, and it's what I do for all the programs. Note that `p1` is a reference to the function, while `p1()` calls the function with no input arguments. The function terminates with the `return` statement, which in this case defines its return value as the figure object.

To avoid overwhelming you, I've boiled down the most important things you need to know right now to make sense of Program 1:

- The `using` keyword imports a package.
- Square brackets create an array from what's inside them.
- You create a range with the syntax `start:step:stop`.
- Greek letters, superscripts, subscripts, and symbols can be parts of variable and function names.
- You can write, for example, `4x` to mean `4 * x`.
- Loops have a local variable scope that can overlap with the enclosing scope.
- Use a period to vectorize any function, i.e., apply it elementwise to an array.

- Use `@.` to vectorize everything that follows.
- Indexing starts at 1.[1] The last index is the keyword `end`.
- Keyword function arguments come after positional ones.

Some detailed explanations on each of these items follow.

Importing packages

Tip
The `using` keyword imports a package. So does `import`, with different implications on how things are referred to by name.

Let's begin with the opening lines,

```
using CairoMakie, LaTeXStrings
using ToeplitzMatrices
```

Only a small core of functionality is loaded into Julia at startup. To get more, we need to load packages via `using` or `import`. The first line, for example, loads the `CairoMakie` and `LaTeXStrings` packages, provided they have been installed. (Capitalization matters.)

Each package creates its own namespace that holds all the objects it defines. You can access anything defined by the package by addressing it with the name of the package, as in `CairoMakie.Figure`. When you load via `import`, that is the only access you get. In addition, though, most packages also export some names into the default `Main` workspace, if you load them via `using`. That's why in Program 1, we can simply call `Figure`.

A package needs to be loaded only once per session. For clarity, I will be explicitly loading all packages needed by each program, so that you can see the dependencies.

The `CairoMakie` package is part of one of the two most popular systems for making plots in Julia, called Makie. (The other popular choice is Plots.) Makie has a few different backends for writing output in different contexts; CairoMakie is the most print-friendly variant.

Finally, a meta-comment: I've created the `SpectralMethodsTrefethen` Julia package, and it defines all the functions I'm presenting. So, you can simply import that package and then run `p1()` to get its output.

Comprehensions, ranges, and vectors
On line 5 we have

```
N = [2^j for j in 3:12]
```

This syntax is known as a comprehension, which will be familiar to Python veterans. Essentially, it's a compact way of writing a loop, with the body of the loop given first. The enclosing square brackets indicate that the result will be an array. The expression `2^j` is evaluated for each value of `j` in the range `3:12`, resulting in a one-dimensional array with entries $2^3, 2^4, \ldots, 2^{12}$.

Arrays are a major part of base Julia. The array type is called `Array`, and it is parameterized by the type of its entries and the number of dimensions:

```
N = [2^j for j in 3:12]
typeof(N)
```

[1] While starting at index 1 is the behavior of standard arrays, there are packages that create array-like objects that can be indexed more arbitrarily. The absolute safest way to get the first index in general is to use the `begin` keyword. See also the help on `axes` and `eachindex`.

As you see here, a `Vector` is an alias for a one-dimensional array. As you might guess, `Matrix` is an alias for a 2-D array.

Line 6 creates an empty array called `errors` that will be used to accumulate approximation errors in the loop.

Syntactic sugar

At line 7 we see

```
h = 2 \pi / N
```

The symbol π is a built-in constant in Julia. This is a good place to point out one cosmetic but lovely feature of Julia: it allows Unicode characters. You can enter π by typing `\pi` followed immediately by . (Personally, I use Greek letters and other symbols so often that I have many alt-key combinations mapped to them.) If you can't be bothered to type π, you can instead write `pi`—but that isn't true in general. For instance, σ is not the same as `sigma`.

Also notice above that we write 2π without a multiplication sign, just as you would mathematically. You see this implied multiplication in line 10 as well, with `2sin(x)`. This won't work for multiplying two variables together, though: e.g., `xy` is a valid variable name, so you would need to use `x * y`.

Julia was written with mathematics in mind from the start, and it is full of small delights like these.

Loops

The opening of the loop in line 7 looks curious:

```
for N in N
```

The loop variable `N` is local to the loop and distinct from the `N` in its enclosing scope. It will be assigned each value within the outer `N` in turn, and then vanish when the loop ends, all without having any effect on the outer value of `N`. I use `for N in N` to avoid having a different name such as `N_values` for the vector.

This aspect of Julia can take getting used to. For example, the following code snippet, on its own, causes an error:

```
for i in 1:5
    n = 2i
end
println("n = ", n)    # error, n is not defined here
```

Since `n` went out of scope at the end of the loop, it is undefined when the print statement executes. You can work around this by declaring `n` to be `global` within the loop, or by giving `n` an outer value before the loop starts.

Vectorization

In line 9, we see a macro, which is signaled by a name starting with the `@` character. A macro runs at compile time and transforms the code following it before it is ever executed. Here, the `@.` macro is a convenient way to vectorize an expression.

Mathematical functions such as `sin` and addition are intended to be applied to scalar values. In scientific computing, though, we often find ourselves wanting to apply a function to every element

of a vector or other array. In MATLAB and NumPy, most predefined mathematical functions will accept an array as input and do exactly that. Julia does *not*, so the expressions `sin([0, π/2, π])` and `1 + [1, 2, 3]` give errors. This can be frustrating, though such errors may expose bugs.

But there is a huge upside. You can put a period `.` at the end of a function name (or in front of an infix operator like `+`) to have it be interpreted elementwise. Unlike MATLAB and NumPy, this works for *any* function, including one you define.

It gets better. Suppose you vectorize an expression to get something like

```
sin.(x).^2 . - 1
```

The Julia parser knows to fuse these vectorizations together into a single loop, rather than creating a vector for `sin.(x)`, another vector for squaring, and a third for subtracting 1. This is great for performance, but it's not so easy on the eyes. That is what `@.` is for; it implies vectorization wherever needed:

```
@. sin(x)^2 - 1
```

The simplicity and speed of vectorization is one of Julia's greatest strengths for scientific computing.[2]

Array indexing

Line 13 constructs a vector of N zeros. Then, in line 14, we get

```
col[[2, 3, N -1, N]] = [ -2/3, 1/12, -1/12, 2/3] / h
```

This is a way to assign values to specific entries of the vector `col`. The left-hand side refers to elements at indices 2, 3, $N-1$, and N, and the right-hand side is a 4-vector as well.

Keyword arguments

Line 25 creates a 2-D Axis object for plotting:

```
ax = Axis(fig[1, 1]; title="Convergence of fourth -order finite differences",
    xscale=log10, yscale=log10, xlabel=L"N", ylabel="error")
```

We'll see later that indexing the figure object lets us create grid layouts. After the first argument, the semicolon indicates that the remaining arguments are given by keyword rather than position. (You can often use a comma instead, but the semicolon is never wrong.) Here, the keywords describe aspects of how the axis object is to look.

Finally, know that the label on the x axis is formatted by LaTeX thanks to the `L` before the opening quote.[3]

0.1.2 Program 2

```
using CairoMakie, LaTeXStrings, ToeplitzMatrices
"p2 - convergence of periodic spectral method (compare p1)"
function p2()
    # For various N (even), set up grid as before:
    N = 2:2:100
    errors = zeros(size(N))
    for (k, N) in enumerate(N)
        h = 2 \pi / N
```

```
        x = [ - \pi + j * h for j in 1:N]
        u = @. exp(sin(x))
        uprime = @. cos(x) * u

        # Construct spectral differentiation matrix:
        col = [0.5 * ( -1)^i * cot(i * h / 2) for i in 1:N -1]
        D = Toeplitz([0; col], [0; reverse(col)])

        # Compute max(abs(D*u - uprime)):
        error = maximum(abs, D * u - uprime)
        errors[k] = error
    end

    fig = Figure()
    ax = Axis(fig[1, 1]; title="Convergence of spectral differentiation",
        xscale=log10, yscale=log10, xlabel=L"N", ylabel="error")
    scatter!(ax, N, errors)
    return fig
end
```

Output 2

```
p2()
```

In line 20 of Program 1, I used `push!` to append a new value to the end of the `errors` array on each iteration of the loop. Technically, this is a tiny bit wasteful, since we have to allocate new space each time through the loop. Since we know in advance that every iteration will contribute exactly one new value, a cleaner implementation is used in Program 2; in line 6, all the space needed for the error vector is allocated before the loop starts.

Mutating functions

We've seen some functions, like `push!` and `scatter!`, whose names end with the bang character, `!`. This is a convention in Julia that the function could be mutating its arguments—changing their values or states. Most plotting commands in Makie, for instance, have both a non-mutating version (e.g., `scatter`), which will create a new figure and axis, and a mutating version (e.g., `scatter!`), which will modify an existing axis.

`enumerate`

Since we abandoned the `push!` strategy to record the errors, we now need to track in the loop the index where the next error value is to be stored. The `enumerate` function is a convenient way to get both the index and the value of each entry in a collection. The syntax `(k, N)` indicates a tuple of two values. By putting the tuple on the left-hand side of an assignment, we destructure the tuple on the right-hand side, as in the following:

```
a, b, c = (1, 2, 3)
a + b + c
```

The main functional difference between a vector and a tuple is that a vector is mutable, so its entries can be changed, while a tuple is immutable.

Important
Row dimension is the first dimension. Separate columns with spaces and rows with semicolons.

Array concatenation
The syntax `[0; col]` in line 11 is worth a mention. Inside square brackets, commas separate entries of a vector. Because the elements of a vector can be anything, including other vectors, this implies that we cannot use commas to splice scalars and vectors into a single vector the way we do with scalars alone:

```
col = [1, 2, 3]    # a vector of integers
[0, col]           # a vector of two entries: 0 and col
```

Instead, we use a semicolon to get a vertical concatenation of the scalar and the vector. Note that the following are equivalent:

```
[0; col]
vcat(0, col)
cat(0, col, dims=1)    # row dimension is the first dimension
```

0.2 Chapter 2

0.2.1 Program 3

```
using CairoMakie
"p3 - band -limited interpolation"
function p3()
    h = 1
    xmax = 10
    x = -xmax:h:xmax                      # computational grid
    xx = -xmax -h/20:h/10:xmax+h/20        # plotting grid
    sinc(t) = iszero(t) ? 1.0 : sinpi(t / h) / ( \pi * t / h)
    funs = [
        x -> float(x==0),
        x -> float(abs(x) <= 3),
        x -> max(0, 1 - abs(x)/3)
        ]
    fig = Figure()
    for (i, f) in enumerate(funs)
        v = f.(x)
        ax = Axis(fig[i, 1], yticks=[0, 1], limits=( -xmax, xmax, -0.5, 1.5))
        scatter!(ax, x, v)
        p = sum( v[j] * sinc.(xx . - x[j]) for j in eachindex(x) )
        lines!(ax, xx, p)
    end
    return fig
end
```

Among other new elements, this is our first subplot grid.

Output 3

```
using SpectralMethodsTrefethen
p3()
```

Function definition
In line 8 we see a new way to define a function, much like one would do it in writing. Another example:

```
r(x) = 1 / (25x^2 + 1)
```

Caution
Once a name is assigned to a function, it cannot be used as a variable name in the same scope.

In lines 10–12 I use an anonymous function (i.e., lambda) syntax

```
x -> float(x==0)
```

Ternary operator
The ternary operator in line 8

```
p ? A : B
```

evaluates to `A` if `p` is true; otherwise, `B`. This device is best used with short expressions, to keep the logic readable at a glance. Some style guides recommend against it entirely.

`sinpi`

The `sinpi`, `cospi`, and `tanpi` functions not only save a multiplication by π, but they can be more accurate, too.

Numeric type conversion

In lines 10 and 11 I use `float` to convert the logical results of type `Bool` to floating-point numbers. The explicit conversions aren't strictly necessary here, but they improve clarity. A more general syntax for type conversion is `convert(T, x)`, which converts `x` to type `T`.

`max` vs. `maximum`

From line 12, note that `max` chooses the largest of its arguments. This differs from previously-seen `maximum`, which returns the largest element of an array.

Subplots

Line 17 shows how to use indexing of a figure object to create subplots. We'll be seeing a lot of that as we go. You can even have grids within subplot grids to construct complicated layouts.

Generators

Line 19 contains more than it might seem. The goal is to evaluate a truncation of the sum in (2.10),

$$\sum_m v_m S_h(x - x_m), \tag{1}$$

for all values of x in the plotting range `xx`. The approach taken here is to evaluate each term at all values of x and sum the resulting vectors. While the syntax resembles a comprehension, it is instead a more primitive generator. The distinction is clearer in a simple example:

```
sum( 1/factorial(n) for n in 0:8 )      # generator

sum( [1/factorial(n) for n in 0:8] )    # comprehension
```

These statements perform the same computation, but the former just runs a loop, while the second constructs a vector from the loop and then sums the entries of the vector.

0.3 Chapter 3

0.3.1 Program 4

```
using CairoMakie, Printf, ToeplitzMatrices
"p4 - periodic spectral differentiation"
function p4()
    # Set up grid and differentiation matrix:
    N = 24
    h = 2 \pi / N
    x = h * (1:N)
    col = [0.5 * ( -1)^j * cot(j * h / 2) for j in 1:N -1]
    D = Toeplitz([0; col], [0; reverse(col)])

    fig = Figure()
    xticks = MultiplesTicks(5, \pi, " \pi")

    # Differentiation of a hat function:
    v = @. max(0, 1 - abs(x - \pi) / 2)
    Axis(fig[1, 1]; title="function", xticks)
    scatterlines!(fig[1, 1], x, v)
    Axis(fig[1, 2]; title="spectral derivative", xticks)
    scatterlines!(fig[1, 2], x, D * v)

    # Differentiation of exp(sin(x)):
    v = @. exp(sin(x))
    vprime = @. cos(x) * v
    Axis(fig[2, 1]; xticks)
    scatterlines!(fig[2, 1], x, v)
    Axis(fig[2, 2]; xticks)
    scatterlines!(fig[2, 2], x, D * v)
    str = @sprintf("max error = %0.4e", maximum(abs, D * v - vprime))
    text!(fig[2, 2], 2.2, 1, text=str, fontsize=12, align=(:left, :bottom))
    return fig
end
```

By now you should be seeing some familiar motifs in Program 4. But there's new stuff to look at, too.

Output 4

```
using SpectralMethodsTrefethen
p4()
```

Formatting strings

In Program 4 we load a new package, `Printf`. It is part of the standard library, so it doesn't require installation. It provides the `@sprintf` macro, which enables C-style formatting of strings. You can see it in action in line 27, where `%0.4e` means to format the following number in scientific notation with four significant digits.

`reverse`

In line 8 you see the use of `reverse` to reverse the order of a vector. You can specify one or more dimensions for reversing an array by using the `dims` keyword argument.

`MultiplesTicks`
Line 11 shows off a Makie ability to create axes ticks that are multiples of any value. The call `MultiplesTicks(5, π, "π")` requests 5 ticks at multiples of π, and the tick labels will have π appended to them. This can be a nice refinement for plotting trigonometric functions.

Keyword shorthand
Remember that arguments after the semicolon must be keyword arguments. In the calls to `Axis`, such as:

```
Axis(fig[1, 1]; title="function", xticks)
```

The `xticks` by itself after the semicolon is shorthand for `xticks=xticks`. When there are many keyword arguments, this trick is a nice way to shorten function calls.

`scatterlines`
Line 16 introduces `scatterlines!`, which combines a scatter plot (markers) with a lines plot (connecting segments).

Symbols
In line 28, the `align` keyword is given a tuple of two values whose names each start with a colon, `:`. This is the syntax for creating a symbol. You can think of a symbol as a non-numeric constant, or as a string that is not meant to be printed.

0.3.2 Program 5

```
using CairoMakie, Printf, FFTW
"p5 - repetition of p4 via FFT (real data only)"
function p5()
    # Differentiation of a hat function:
    N = 24
    h = 2 \pi / N
    x = h * (1:N)
    function fftderiv(v)
        N = length(v)
        v = rfft(v)
        w = im * [0:N/2 -1; 0] .* v
        return irfft(w, N)
    end

    fig = Figure()
    xticks = MultiplesTicks(5, \pi, " \pi")

    # Differentiation of a hat function:
    v = @. max(0, 1 - abs(x - \pi) / 2)
    w = fftderiv(v)
    Axis(fig[1, 1]; title="function", xticks)
    scatterlines!(fig[1, 1], x, v)
    Axis(fig[1, 2]; title="spectral derivative", xticks)
    scatterlines!(fig[1, 2], x, w)
```

```
    # Differentiation of exp(sin(x)):
    v = @. exp(sin(x))
    vprime = @. cos(x) * v
    w = fftderiv(v)
    Axis(fig[2, 1]; xticks)
    scatterlines!(fig[2, 1], x, v)
    Axis(fig[2, 2]; xticks)
    scatterlines!(fig[2, 2], x, w)
    str = @sprintf("max error = %0.4e", maximum(abs, w - vprime))
    text!(fig[2, 2], 2.2, 1, text=str, fontsize=12, align=(:left, :bottom))
    return fig
end
```

Program 5 is our first look at the `FFTW` package. It provides `fft` and `ifft` that are equivalent to the functions in MATLAB. But it also provides additional tools for getting greater efficiency, when doing so is important.

Output 5

```
p5()
```

Real FFT

The Fourier transform of real data is an even function in wavenumber space. The functions `rfft` and `irfft` take advantage of this symmetry to reduce the number of operations by about a factor of two. Because of an ambiguity about even/odd cases, the inverse transform needs to be given the length of the original data, as in `irfft(w, N)`.

FFT differentiation

Unlike in *SMiM*, I've chosen to put the operation of differentiation via FFT into a short function starting at line 8. Until recently, it was impossible to define a multiline function at an arbitrary point within a MATLAB script, so it was easier to simply repeat short motifs as needed. But it's better practice to use functions for this: it clarifies the purpose of the code, makes it more reusable, and eliminates bugs due to copy–paste errors or customizations. In Julia, this structure can also improve performance.

Hat accent

In math, including *SMiM*, it's conventional to put a hat over the name of a function to indicate its Fourier transform. In Julia, you can type to get `v`, for example. Note that the hat is just a character, and it doesn't have any special meaning to Julia.

Such accents can be hard to read in some fonts, so this practice comes down to personal preference.

Imaginary unit

In line 11 we see a reference to `im`, the imaginary unit. In Julia, `im` is a built-in constant with the value $\sqrt{-1}$. You can also write `1im` or `2im`, for example, to get i or $2i$. Notice that just as Julia draws distinctions between integers and floating-point numbers, it also distinguishes between real and complex numbers:

```
println("Type of 1 is ", typeof(1))
println("Type of 1.0 is ", typeof(1.0))
println("Type of 1im is ", typeof(1im))
println("Type of 1.0im is ", typeof(1.0im))
```

Mixing numeric types

Operations involving mixed numeric types try to promote the operands to a common type. This often goes without notice. But attempting a type incorrectly can produce errors:

```
x = [1, 2, 3]
x[2.0]    # error, can't use floating -point number as an index
```

So can trying to store a value into an array of a less-general type:

```
z = [ \pi, 2 \pi, 3 \pi]
@. z *= 1im    # error, can't store complex in a real array
```

Tip

When you see an `InexactError`, it may be the result of a numeric type that can't be used in context without changing its value.

0.3.3 Program 6

```
function p6(N=128, tmax=8, \Deltat= \pi/2N)
    # Grid, variable coefficient, and initial data:
    h = 2 \pi / N
    x = h * (1:N)
    function fftderiv(v)
        N = length(v)
        v = rfft(v)
        w = im * [0:N/2 -1; 0] .* v
        return irfft(w, N)
    end

    c = @. 0.2 + sin(x - 1)^2
    v = @. exp( -100 * (x - 1) .^ 2)
    vold = @. exp( -100 * (x - 0.2 \Deltat - 1)^2)

    # Time -stepping by leap frog formula:
    tplot = 0.15
    plotgap = round(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)
    data = [v zeros(N, ntime)]
    t = \Deltat * (0:ntime)
    for i = 1:ntime
        w = fftderiv(v)
        vnew = vold - 2 \Deltat * c .* w
        data[:, i+1] = vnew
```

```
        vold, v = v, vnew
    end
    # Plot results:
    return heatmap(x, t[1:plotgap:end], data[:, 1:plotgap:end];
        colormap=:amp,
        axis=(xlabel=L"x", xticks=MultiplesTicks(5, \pi, " \pi"), ylabel=L"t"))
end
```

Our first PDE solution! But the output here is a bit unlike Output 6 in *SMiM*.

Output 6

```
p6()
```

The `waterfall` plot in MATLAB is not natively in Makie. Rather than try to reconstruct it, I have opted to use a `heatmap` instead, which conveys the result equally well, if a bit less lusciously. In fact, you can more clearly see a flaw in the solution; more on that in the next section.

Documentation string
Line 2 demonstrates a documentation string for a function. A string value immediately preceding a function keyword provides a string that is printed out at the prompt when you enter `?` followed by the function name.

`round`
Note in lines 21 and 23 that the default output type of `round` is the same as the input type, so to use the result as an array index, we need to convert it to an integer with `round(Int, ...)`.

Horitontal concatenation
In line 24, I use square brackets to construct an array, with horizontal concatenation implied by the space. In Julia, vectors are one-dimensional arrays, but when a vector is used in a matrix context, it is treated as a column vector. Thus, the result of `[v zeros(N, ntime)]` is an array whose first column is `v` and whose remaining `ntime` columns are zeros. You could also use `hcat` or `cat(.., dims=2)` for horizontal concatenation.

0.3.4 Animated alternative to Output 6

One advantage I have here over a printed book is the use of animations. Accordingly, here is an alternative version of Program 6 that produces an animation of the solution as it evolves in time.

```
using CairoMakie, LaTeXStrings, FFTW
"p6anim - variable coefficient wave equation"
function p6anim(N=128, tmax=8, \Deltat= \pi/2N)
    h = 2 \pi / N
    x = h * (1:N)
    function fftderiv(v)
        N = length(v)
        v = rfft(v)
        w = im * [0:N/2 -1; 0] .* v
        return irfft(w, N)
    end
```

```
    c = @. 0.2 + sin(x - 1)^2

    # Time -stepping by leap frog formula:
    ntime = round(Int, tmax / \Deltat)
    \Deltat = tmax / ntime
    time = Observable(0.0)
    v = Observable(@. exp( -100 * (x - 1) .^ 2))
    title = @lift latexstring(@sprintf("\$t = %0.2f\$", $time))
    vold = @. exp( -100 * (x - 0.2 \Deltat - 1)^2)
    fig = lines(x, v;
        axis=(; xlabel=L"x", xticks=MultiplesTicks(5, \pi, " \pi"), title))
    anim = record(fig, "p6anim -$N -$tmax.mp4"; framerate=60) do io
        recordframe!(io)
        for n in 1:ntime
            w = fftderiv(v[])
            vnew = vold - 2 \Deltat * c .* w
            vold = v[]
            time[] = n * \Deltat
            v[] = vnew
            recordframe!(io)
        end
    end
    return anim
end
```

The return value of this function is the file name of the resulting animation, which is a video file in MP4 format.

Output 6-anim

```
p6anim(128, 12)
```

Both Output 6 and the animation show a small amount of energy traveling coherently in the wrong direction. I'll let you think about the source of that error. Leapfrog time-stepping is notable for propagation without numerical dissipation, so the error just goes on its merry way. Many other time-stepping methods introduce some artificial dissipation, which could make the solution better-looking—but not necessarily more accurate.

Animation with Makie

The animation mechanism in Makie is to define so-called Observable quantities whose changing values drive the animation. In Program 6-anim, there are two Observable values: scalar `time` and the discrete solution `v`. Other variables whose values are derived from an Observable quantity should be defined using the `@lift` macro, as in line 21 to define `title`. Plot objects created using Observable values, or values lifted from them, will be updated whenever the underlying Observable is changed. Thus, our `lines` object in lines 23–24 depends both on `v` and on `title`, which in turn depends on `time`.

The animation is created by a call to `record`. The only remaining quirk is that when you want to get or change the value of an Observable, you have to dereference it by adding empty square brackets to the name. You can see I have done this to every use of `v` and `time` within the recording loop.

`latexstring`

One more fine point in this code is the use of `latexstring` in line 21. This is another function from the `LaTeXStrings` package. We can't use the `L"` string macro here, because we need to LaTeX-ify a string whose value is unknown at compile time due to the interpolation of the time value. Note that the `\$` is needed in the string to get a literal dollar sign to designate LaTeX math mode.

String interpolation

In line 24, I create a file name with the string `"p6anim-$N-$tmax.mp4"`. The syntax `$(...)` is used to interpolate the values of variables into a string. When the expression to be interpolated is just a variable name, you can omit the parentheses, as with `$N` and `$tmax`.

0.4 Chapter 4

0.4.1 Program 7

```
using CairoMakie, LaTeXStrings
using ToeplitzMatrices, ForwardDiff
"p7 - accuracy of periodic spectral differentiation"
function p7(Nmax = 50)
    # Compute derivatives for various values of N:
    N = 6:2:Nmax
    funs = [x -> abs(sin(x))^3            x -> exp( -sin(x / 2)^( -2));
            x -> 1 / (1 + sin(x / 2)^2)   x -> sin(10x)]
    E = zeros(2, 2, length(N))
    for (k, N) in enumerate(N)
        h = 2 \pi / N
        x = h * (1:N)
        col = [0.5 * ( -1)^j * cot(j * h / 2) for j in 1:N -1]
        D = Toeplitz([0; col], [0; reverse(col)])
        for (i, f) in pairs(funs)
            v = f.(x)
            vprime = ForwardDiff.derivative.(f, x)
            E[i, k] = maximum(abs, D * v - vprime)
        end
    end

    # Plot results:
    titles = [L"|\sin(x)|^3" L"\exp( -\sin^{ -2}(x/2))"; L"1/(1+\sin^2(x/2))" L"\sin(10x)"]
    fig = Figure()
    ax = [Axis(fig[i, j]; title=titles[i, j], yscale=log10) for i in 1:2, j in 1:2]
    linkaxes!(ax...)
    for (i, ax) in pairs(ax)
        scatterlines!(ax, N, E[i, :])
    end
    ax[1, 1].ylabel = ax[2, 1].ylabel = "error"
    ax[2, 1].xlabel = ax[2, 2].xlabel = L"N"
    return fig
end
```

One of my favorite perks in Makie is the `linkaxes!` function, which tells Makie to give all the linked axes the same limits. That's used to great effect in the output (like in the original), where it's instantly clear how much slower convergence is in the top two cases:

Output 7

```
using SpectralMethodsTrefethen
fig = p7()
```

That visual story is missing if you let each individual plot choose its own limits, as in the following:

```
map(a -> empty!(a.yaxislinks), fig.content)
fig
```

Abstract types

We've worked with Toeplitz matrices a few times. Here's a baby one.

```
using ToeplitzMatrices
col = [0; -2; 6; zeros(3); -6; 2]
A = Toeplitz(col, -col)
```

You might be surprised that a `Toeplitz` matrix isn't actually a matrix!

```
A isa Matrix
```

However, it does behave like an array in many ways. For instance, we can get its size:

```
size(A)
```

We've multiplied Toeplitz matrices with vectors, too. Julia has a special way of indicating that an object is matrix-like, even when it is not truly represented by a contiguous block of memory:

```
A isa AbstractMatrix
```

Being typed as an `AbstractMatrix`, the object `A` promises that it can do the basic things a matrix should do. But behind the scenes, special algorithms can be used to save time and memory.

A similar situation holds with ranges:

```
N = 6:2:50
println("N is a Vector? ", N isa Vector)
println("N is an AbstractVector? ", N isa AbstractVector)
```

Thus, while we can read an element of the range, we can't overwrite one:

```
@show N[4]
N[1] = 2    # error, not allowed
```

If you want to convert a range into a proper vector, you can apply `collect` to it. Conversion also happens automatically in some situations, like a vertical concatenation:

```
println("collect(N) is a Vector? ", collect(N) isa Vector)
@show [1; N];
```

Another such situation arises in line 28 of Program 7. Since exponentiating the range `(-15:5:0)` creates an output that is not evenly spaced, the result is a vector.

```
println("10.0 ^ ( -15:5:0) is a Vector? ", 10.0 .^ ( -15:5:0) isa Vector)
println("10 * ( -15:5:0) is a Vector? ", 10 * ( -15:5:0) isa Vector)
```

Splatting

In the call to `linkaxes!` on line 26, we see the syntax `ax...`, which is called splatting. This is a convenient way to indicate that all the entries of an iterable object are to be included as function arguments.

`pairs`

In lines 15 and 27 we have loops set up using `pairs`, like

```
for (i, f) in pairs(funs)
```

The `pairs` function is similar to `enumerate`, in that it returns a joint iterable for the indexes and values of an array. However, `pairs` uses an index that is native to its argument, whereas `enumerate` always returns a range starting at 1.

Program 7 is set up so that `funs`, `titles`, and `ax` are all 2×2 in the same way we want them to appear in the final subplot grid. So, when we call `pairs(funs)` or `pairs(ax)`, we get an iterator whose index is a `CartesianIndex` value, like in the following:

```
A = [11 12; 21 22]
for (i, v) in pairs(A)
    println("i = $i, v = $v, A[i] = $(A[i])")
end
```

Because `i` is a `CartesianIndex`, we can use it to index into `A` directly, as in `A[i]`. Even better, we can use it as the *first two* indices into the 3-D array `E`, and then give a third index. This is incredibly convenient syntax for working with multidimensional arrays.

Automatic differentiation

The `ForwardDiff` package provides a convenient way to compute derivatives called automatic differentiation. It applies the basic rules of differentiation, like the product rule and chain rule, to compute the derivative of a Julia function at a given value of its argument. Even though it gives floating-point results, it is not a numerical approximation, but rather an exact application of the differentiation rules.

Automatic differentiation is a highly developed area for Julia, because of the benefits it offers to scientific computing (e.g., Jacobian and Hessian matrices) and machine learning (e.g., back-propagation).

Chained assignment

In each of lines 30 and 31 you can see two equals signs. This is a chained assignment, which allows you to assign the same value to multiple variables in one line. They are evaluated right to left, and since the result of an assignment statement is the value that was assigned, so, for instance, the value of `ax[2, 1].ylabel` is the string `"error"`, which is then assigned to `ax[1, 1].ylabel`.

You can also chain comparisons, like `1 <x <= 2`, or `-1 .<A .<1`.

0.4.2 Program 8

```
using LinearAlgebra, ToeplitzMatrices
"p8 - eigenvalues of harmonic oscillator -u + xš u on "
function p8(L = 8)                          # domain is [ -L, L], periodic
    for N in 6:6:36
        h = 2 \pi / N
        x = h * (1:N)
        x = L * (x . - \pi) / \pi
        column = [ - \pi^2 / 3h^2 - 1/6; [ -0.5 * ( -1)^j / sin(h * j / 2)^2 for j in 1:N -
1]]
        Dš = ( \pi / L)^2 * Toeplitz(column, column)    # 2nd -order differentiation
        \lambda = eigvals( -Dš + Diagonal(x.^2))
        @show N
        println.( \lambda[1:4])
        println()
    end
```

```
    return nothing
end
```

This is our first use of the `LinearAlgebra` package, part of the standard library. While matrix multiplication and linear system solving are in base Julia, most other linear algebra functionality is in this package, including norms, factorizations, and singular values, and eigenvalues.

Output 8

```
p8()
```

Eigenvalues

Since we only want eigenvalues here, we use `eigvals` to get them. This is faster than getting both the eigenvalues and eigenvectors, which is done by `eigen`. In either case, the default is to return everything lexicographically sorted by real and imaginary parts of the eigenvalues; you can override this by adding, for example, `sortby=abs` to use magnitude (i. e., absolute value in the real case) instead.

Tip

If you want to use the Unicode character λ, remember that the English spelling of this letter is "lambda", with a silent "b".

Diagonal matrix

The `Diagonal` constructor creates a special type of matrix that stores only the diagonal elements:

```
using LinearAlgebra
Diagonal(1.0:5)
```

As such, it is an `AbstractMatrix`, not a `Matrix`. This allows other functions to specialize their behavior to take advantage of the diagonal structure. If you want a true `Matrix` constructed from its diagonal, use `diagm`:

```
diagm(1.0:5)
```

This would be necessary in order to change the off-diagonal elements, for example. Other functions will see this as a generic matrix without special structure.

0.5 Chapter 5

0.5.1 Program 9

```
using CairoMakie, Printf, Polynomials, SpectralMethodsTrefethen
"p9 - polynomial interpolation in equispaced and Chebyshev pts"
function p9(N = 16)
    xx = range( -1, 1, 801)
    fig = Figure(size=(640, 300))
    cases = [
        ("equispaced points", [ -1 + 2n / N for n in 0:N]),
        ("Chebyshev points", [cospi(n / N) for n in 0:N]),
    ]
    f(x) = 1 / (1 + 16x^2)
    for (i, (title, x)) in enumerate(cases)
        v = f.(x)
        ax = Axis(fig[1, i]; title, limits=(nothing, ( -1, 1.5)))
        scatter!(ax, x, v)
        u = fit(Polynomial, x, v)       # interpolation
        uu = u.(xx)                     # evaluation of interpolant
        lines!(ax, xx, uu)
        maxerr = @sprintf("%.5g", maximum(abs, uu - f.(xx)))
        text!(ax, 0, -0.5; text="max error = $maxerr", align=(:center, :bottom))
    end
    return fig
end
```

In *SMiM*, the notational conventions shifted a little bit for this code. Rather than using u to mean a function with grid values called v, `p9.m` uses the variable u to mean grid values of a polynomial p. That's consistent with the math in this chapter, but not with the code in the preceding ones, and I've opted in Program 9 to stay with `u` to mean the function whose grid values are `v`. We'll see that in future programs as well.

On a subtle but important mathematical note, you would find big differences in errors between my Program 9 and the original in *SMiM* at larger values of N. The `polyfit` function in MATLAB (at this writing time) computes the coefficients of the interpolating polynomial in the monomial basis, which is a bad choice due to its poor conditioning. The algorithm in line 15 above, though, uses a better-conditioned approach from **?**. Of course, while that computes the interpolating polynomial accurately, the polynomial is not necessarily an accurate representation of the underlying function when the nodes are equally spaced!

Output 9

```
using SpectralMethodsTrefethen
p9()
```

> **`range`**
> In line 4 we see a new way to create a range, with the syntax `range(begin, end, number)`. This is the analog of `linspace` in MATLAB and NumPy.

> **Polynomials**
> The Polynomials package provides ways to work with polynomials.

Important
Unlike in MATLAB, coefficients should be given in order of increasing degree:

```
using Polynomials
Polynomial([1, 0, -2.5, 5])
```

In this case, we aren't constructing polynomials directly. Rather, I use its `fit` function to perform polynomial interpolation, analogously to `polyfit` in MATLAB. More generally, both functions do least-squares fitting if given a polynomial degree less than the lengths of the data vectors.

Enumerating tuples
Note the syntax used with `enumerate` in line 11. Since the enumerated object is a vector whose elements are tuples, those tuples are destructured using the nested syntax `(i, (title, x))`.

Semi-automatic axis limits
In line 13 we see an axis being created with `limits=(nothing, (-1, 1.5))`. This syntax means to let the x limits be determined automatically, while y is restricted to $[-1, 1.5]$.

0.5.2 Program 10

```
using CairoMakie, Polynomials
"p10 - polynomials and corresponding equipotential curves"
function p10(N = 16)
    fig = Figure()
    cases = (
        ("equispaced points", [ -1 + 2n / N for n in 0:N]),
        ("Chebyshev points", [cospi(n / N) for n in 0:N]),
    )
    for (i, (title, x)) in enumerate(cases)
        p = fromroots(x)

        # Plot p(x) over [ -1,1]:
        scatter(fig[i, 1], x, zeros(size(x)), axis=(;title))
        lines!(fig[i, 1], -1..1, x -> p(x))

        # Plot equipotential curves:
        xx, yy = -1.3:0.02:1.3, -1:0.02:1
        pp = [p(x + im * y) for x in xx, y in yy]
        zz = @. log10(max(abs(pp), 1e -16))
        ax = Axis(fig[i, 2]; title, aspect=DataAspect())
        contour!(ax, xx, yy, zz; levels= -4:0, linewidth=2,
            colormap=:atlantic)
        scatter!(ax, x, zeros(size(x)); color=:black)
    end
    return fig
end
```

This is our first contour plot.

Output 10

```
p10()
```

`fromroots`
Line 10 uses `fromroots` to construct the monic polynomial with roots at the specified locations.

Plotting a function
In line 14 I introduce a tidy way to plot most functions without providing an explicit discretization of them. The syntax `lines(a..b, f)` will automatically select points in the interval $[a, b]$ to produce a smooth plot of the function f. This is analogous to `fplot` in MATLAB.

Avoiding log of zero
In line 19 I use `max` to make sure that the resulting logarithm is finite. Values of `-Inf` can confuse the contour plot.

Aspect ratio
In line 20, I have created an axis with specified aspect ratio. This is important in many situations, including the complex plane, where you want a unit in the x and y directions to have the same physical lengths in the plot. This turns out to be a surprisingly complicated subject for plotting, depending on whether you want to adjust the axes limits (as done here with `DataAspect()`) or the shape of the axis box.

Contour plot
Line 20 creates the contour plot. The `levels` keyword can be an integer, to get the specified number of equally spaced levels, or a vector of explicit values. The `colormap` specifies how contour levels map to colors. You have a lot of choices.

0.6 Chapter 6

0.6.1 cheb

```
"""
    cheb(N)
Chebyshev differentiation matrix and grid.
"""
function cheb(N)
    (N == 0) && return zeros(1, 1), [1.0]
    x = [sinpi(k / 2N) for k in -N:2:N]
    c = [2; ones(N -1); 2]
    @. c[2:2:end] = -c[2:2:end]
    # compute the off -diagonal entries
    D = [c[i] / (c[j] * (x[i] - x[j] + (i==j))) for i in eachindex(x), j in eachindex(x)]
    # negative sum trick to get the diagonal entries
    for i in axes(D, 1)
        D[i, i] = -sum(D[i, j] for j in axes(D, 2) if j != i)
    end
    return D, x
end
```

This is the first formal function in *SMiM*. There is an algorithmic difference in my implementation on line 7, which computes the Chebyshev points via

$$x_j = -\sin\left(\frac{k\pi}{2N}\right),\ k = -N, -N+2, \ldots, N-2, N. \tag{2}$$

This formula is a trivial trigonometric identity away from the standard cosine expression, but unlike that one, this one produces nodes that are perfectly symmetric around zero in floating-point arithmetic **?**.

Short-circuit logic

Line 6 might look odd. It takes advantage of the short-circuiting logical AND, `&&`. Short-circuiting means that if the first clause is false, then the second clause isn't evaluated, because the entire expression must be false anyway. This is a nice feature to avoid pesky runtime errors in conditional expressions, as in

```
if (length(x) >= 5) && (x[5] > 0)
```

Here, though, it's being used as a compact—some would say cryptic—alternative to the construction

```
if N == 0
    return zeros(1, 1), [1.0]
end
```

A similar trick works with the OR operator `||`, which skips the second clause if the first is true. For example,

```
(max_error < tol) || @warn("Error exceeds tolerance.")
```

Minimizing allocations

There is a subtle difference between the given line 9,

```
@. c[2:2:end] = -c[2:2:end]
```

and the functionally equivalent line

```
c[2:2:end] = -c[2:2:end]
```

The former version implies a copy of the vector from the right-hand side to the left-hand side. But the second version implies an assignment to a subarray of the vector on the left-hand side, thus requiring extra allocation of temporary storage for the subarray.

In fact, even the first version requires allocations (for the right-hand side) we don't need. The fastest version of this line is a loop:

```
for j in 2:2:length(c)
    c[j] = -c[j]
end
```

Coders in MATLAB and Python are (rightly) taught to avoid loops, but in Julia, loops carry no intrinsic penalty—sometimes, quite the opposite.

0.6.2 Program 11

```
using CairoMakie, LaTeXStrings
using SpectralMethodsTrefethen, ForwardDiff
"p11 - Chebyshev differentation of a smooth function"
function p11()
    f(x) = exp(x) * sin(5x)
    df_dx(x) = ForwardDiff.derivative(f, x)
    fig = Figure()
    for (i, N) in enumerate([10, 20])
        D, x = cheb(N)
        v = f.(x)

        Axis(fig[i, 1]; title=latexstring("\$u(x),\\,  N=$N\$"))
        scatter!(fig[i, 1], x, v)
        lines!(fig[i, 1], -1..1, f)

        error = D * v - df_dx.(x)
        Axis(fig[i, 2]; title=latexstring("error in \$u'(x),\\,  N=$N\$"))
        scatterlines!(fig[i, 2], x, error; linestyle=:dot)
    end
    return fig
end
```

Output 11

```
using SpectralMethodsTrefethen
p11()
```

There are no new Julia tricks here, but there is a stylistic change from the original Output 11. In both versions, the solid curves on the left column show the given exact function. In the right column, the original program connected the dots with solid lines again, but only as a visual guide—the lines

don't mean anything, really. I've had students get confused on that point, so here I've used a dotted line to connect the points, just to hint that something is different.

0.6.3 Program 12

```
using CairoMakie, LaTeXStrings, SpectralMethodsTrefethen
"p12 - accuracy of Chebyshev spectral differentiation (compare p7)"
function p12(Nmax = 50)
    # Compute derivatives for various values of N:
    # 3rd deriv in BV, C^ \infty; analytic, polynomial
    funs = [x -> abs(x)^3            x -> exp( -x^( -2));
            x -> 1 / (1 + x^2)       x -> x^10]
    E = zeros(2, 2, Nmax)
    for N in 1:Nmax
        D, x = cheb(N)
        for (i, f) in pairs(funs)
            v = f.(x)
            df_dx = ForwardDiff.derivative.(f, x)
            E[i, N] = maximum(abs, D * v - df_dx)
        end
    end

    # Plot results:
    fig = Figure()
    titles = [L"|x|^3"  L"\exp( -x^2)"; L"1/(1+x^2)"  L"x^{10}"]
    ax = [Axis(fig[i, j]; title=titles[i, j], yscale=log10) for i in 1:2, j in 1:2]
    linkaxes!(ax...)
    for (i, ax) in pairs(ax)
        scatterlines!(ax, 1:Nmax, E[i, :])
    end
    ax[1, 1].ylabel = ax[2, 1].ylabel = "error"
    ax[2, 1].xlabel = ax[2, 2].xlabel = L"N"
    return fig
end
```

Output 12

```
p12()
```

You're meant to compare these plots to Output 7 for periodic functions.

0.7 Chapter 7

0.7.1 Program 13

```
using CairoMakie, Printf, LinearAlgebra, SpectralMethodsTrefethen
"p13 - solve linear BVP u_xx = exp(4x), u( -1) = u(1) = 0"
function p13(N = 16)
    D, x = cheb(N)
    Dš = D^2
    Dš = Dš[2:N, 2:N]                    # boundary conditions
    rhs = @. exp(4x[2:N])
    v = Dš \ rhs                         # Poisson eq. solved here
    v = [0; v; 0]
    xx = range( -1, 1, 801)
    u = chebinterp(v)                    # interpolate grid data
    exact(x) = @. (exp(4x) - sinh(4) * x - cosh(4)) / 16;
    maxerr = maximum(abs, u.(xx) - exact.(xx))
    title = @sprintf("max err = %.3g", maxerr)
    fig = lines( -1..1, u; axis=(; title, xlabel=L"x", ylabel=L"u(x)"))
    scatter!(x, v)
    fig
end
```

I have departed from the original `p13.m` in a significant way. Rather than using `polyfit` to interpolate the solution to a finer grid, I have used the barycentric interpolation formula presented in the exercises of Chapter 6. This is a much faster and stabler method, as Trefethen made clear in a **?** published after *SMiM*.

Output 13

```
using SpectralMethodsTrefethen
p13()
```

> **Superscript**
> Why not use a superscript for the second derivative matrix? You get it by typing , then .

Here is my implementation of general barycentric interpolation:

```
"""
    baryweights(x)
Compute the barycentric weights for the nodes in vector ‘x‘. Returns a vector of the
same length as ‘x‘.
"""
function baryweights(x)
    m, M = extrema(x)
    x = 4x / (M - m)    # rescale to avoid overflow/underflow
    idx = eachindex(x)
    return [1 / prod(2(x[j] - x[k]) for k in idx if k !== j) for j in idx]
end

"""
 baryinterp(x, v)
 baryinterp(x, v, w)
Create a polynomial interpolant by the barycentric formula for the function values in
```

```
vector ‘v‘ at the node locations in vector ‘x‘. If given, ‘w‘ is a vector of the
barycentric weights; otherwise it is computed from the nodes. The return value is
a callable function of the interpolation variable.
"""
function baryinterp(x, v, w=baryweights(x))
    numer = denom = 0.0
    return function(s)
        for i in eachindex(x)
            term = w[i] / (s - x[i])
            if isinf(term)
                return v[i]
            end
            numer += term * v[i]
            denom += term
        end
        return numer / denom
    end
end
```

The implementation is split into computation of the weights and application of the formula. This allows the caller to provide precomputed weights if they are available, as is the case for Chebyshev points. (The rescaling in line 8 of `baryweights` is discussed in the paper I cited above.)

Conditional generator

Line 9 shows that a generator can include an `if` statement, which makes the entire weight generation process a one-liner.

Optional arguments

The definition of `baryinterp` in line 20 shows how to define an optional input argument. If only two arguments are given, then `w` is computed by a call to `baryweights`. All earlier arguments are available at this step.

Returning a function

The return value of `baryinterp` is a function, the interpolant. The syntax in line 23 sets this up as an anonymous function. It will have access to all the variables in scope at the time of its defintion, including the enclosing scope.

`nothing`

The `nothing` in line 24 is a special value of type `Nothing`. Nothing equals `nothing` except `nothing`, though the syntax `isnothing` is more idiomatic to check for it. I'm using it here to keep track of whether the given value of `s` is equal to one of the nodes. Mathematically, that creates a L'Hôpital's rule situation, but the code has to check for it explicitly.

`break`

The `break` statement in line 29 is used to exit the loop immediately when a node is found that matches the input `s`. There's no need to finish the loop, since the value of the interpolant is known at this point.

dot

The `dot` function in line 32 is defined in the `LinearAlgebra` package, and it computes the dot product of two vectors. In the complex-valued case, it does the proper inner product by taking the complex conjugate of the first vector.

This is a good opportunity to point out a consequence of the fact that all vectors in Julia have a column orientation. It might surprise you that the following snippet fails:

```
A = ones(3, 3)
A[1, :] * A[:, 3]    # error, both operands are 3 \times1
```

Instead, you could use a conjugate transpose, `A[1, :]'  * A[:, 3]`, or you could just use `dot`.

With `baryinterp` in hand, we can specialize it to the Chebyshev case, where we don't even need to provide the nodes:

```
"""
    chebinterp(v)
Create a callable interpolant for the vector ‘v‘ of values given at Chebyshev nodes in [ -
1, 1].
"""
function chebinterp(v)
    n = length(v) - 1
    wc = [0.5; ( -1).^(1:n -1); 0.5 * ( -1)^n]
    xc = [cospi(k / n) for k in 0:n]
    baryinterp(xc, v, wc)
end

using CairoMakie
_, x = cheb(20)
f(x) = sin(3exp(x))
v = f.(x)
fig = lines( -1..1, chebinterp(v))
scatter!(x, v)
fig
```

0.7.2 Program 14

```
using CairoMakie, LaTeXStrings, SpectralMethodsTrefethen
"""
p14 - solve nonlinear BVP u_xx = exp(u), u( -1) = u(1) = 0 (compare p13)
"""
function p14(N = 16)
    D, x = cheb(N)
    Dš = (D^2)[2:N, 2:N]
    v = zeros(N -1)
    it = 0
    while true              # fixed -point iteration
        vnew = Dš \ exp.(v)
        change = maximum(abs, vnew - v)
        v = vnew
        it += 1
        (change < 1e -15 || it > 99) && break
    end
```

```
    v = [0; v; 0]
    p = chebinterp(v)
    fig = Figure()
    title = "no. steps = $it,  u(0) = $(p(0))"
    ax = Axis(fig[1, 1]; title, xlabel=L"x", ylabel=L"u(x)")
    scatter!(ax, x, v)
    lines!(ax, -1..1, p)
    return fig
end
```

I use `chebinterp` to get the value of the solution at $x = 0$, rather than indexing into the discrete solution directly. This would work just as well for odd values of N, where there is no node at $x = 0$.

Output 14

```
p14()
```

0.7.3 Program 15

```
using CairoMakie, Printf
using SpectralMethodsTrefethen, LinearAlgebra
"p15 - solve eigenvalue BVP u_xx = \lambda u, u( -1) = u(1) = 0"
function p15(N = 36)
    D, x = cheb(N)
    Dš = (D^2)[2:N, 2:N]
    \lambda, V = eigen( -Dš)
    fig = Figure(size=(600, 600))
    ax = [Axis(fig[i, 1], limits=( -1, 1, -0.64, 0.64)) for i in 1:6]
    for (mode, ax) in zip(5:5:30, ax)        # plot 6 eigenvectors
        v = [0; V[:, mode]; 0]
        scatter!(ax, x, v)
        u = chebinterp(v)
        lines!(ax, -1..1, u)
        eig = "eig $mode = $( - \lambda[mode] * 4/ \pi^2) \piš/4"
        text!(ax, 0, 0.45; text=eig, fontsize=12, align=(:center, :baseline))
        PPW = @sprintf("%.1f", 4N / ( \pi * mode))
        text!(ax, 0.7, 0.45; text="$PPW  ppw", fontsize=12, align=(:left, :baseline))
    end
    hidespines!.(ax)
    hidedecorations!.(ax)
    return fig
end
```

Output 15

```
p15()
```

> **Eigenvalues and eigenvectors**
> Since we want both eigenvalues and eigenvectors, we use the `eigen` function rather than `eigvals`.

> **Figure size**
> The `size` keyword in line 9 of `p15` allows us to specify the size of the figure. The exact meaning

of the units is a bit complicated, but my goal here is to get more vertical space than the default of 600 by 450.

`zip`
The `zip` function in line 10 allows us to loop over two identically sized collections at once.

`hidespines!` and `hidedecorations!`
The `hidespines!` and `hidedecorations!` functions in line 17 are used to hide the axes and ticks, which makes the plot look cleaner and matches the original style. Note that they are vectorized to be applied to all the axes at once, though we could have called them from within the loop instead.

0.7.4 Program 16

```
using CairoMakie, Printf, LaTeXStrings
using LinearAlgebra, SpectralMethodsTrefethen
"p16 - Poisson eq. on [ -1,1] x [ -1,1] with u = 0 on boundary"
function p16(N = 24)
    # Set up grids and tensor product Laplacian and solve for u:
    D, x = cheb(N)
    y = x
    f(x, y) = 10sin(8x * (y - 1))
    F = [f(x, y) for x in x[2:N], y in y[2:N]]
    Dš = (D^2)[2:N, 2:N]
    \Delta = kron(I(N -1), Dš) + kron(Dš, I(N -1))    # Laplacian

    fig = Figure(size=(640, 800))
    ax1 = Axis(fig[1, 1]; title="Nonzeros in the Laplacian",
      aspect=1, yreversed=true)
    spy!(ax1, \Delta)
    @time v = \Delta \ vec(F)    # solve problem and watch the clock

    # Reshape long 1D results onto 2D grid (reversing to usual direction):
    V = zeros(N+1, N+1)
    V[2:N, 2:N] .= reshape(v, N -1, N -1)
    icen = div(3N, 4) + 1
    value = V[icen, icen]

    # Interpolate to finer grid and plot:
    xx = yy = range( -1, 1, 201)
    UU = interp2dgrid(V, chebinterp, chebinterp, xx, yy)
    M = maximum(abs, UU)
    ax2 = Axis3(fig[2, 1]; title="Solution of Poisson equation",
      xlabel=L"x", ylabel=L"y", zlabel=L"u(x,y)")
    surface!(ax2, xx, yy, UU; colormap=:redsblues, colorrange=( -M, M))
    str = latexstring("u(2^{ -1/2},2^{ -1/2}) = $(@sprintf("%0.11f", value))")
    text!(ax2, -1, 1, 1.4M; text=str, align=(:left, :top))
    rowsize!(fig.layout, 1, Relative(0.35))
    return fig
end
```

Output 16

```
p16()
```

A lot is going on here! This is our first look at a problem with two space variables. My comments on the interpolation aspect of Program 16 are given in detail below, but first, here are notes on the rest of the program.

Nested loop in a comprehension

The syntax in line 9 is comprehension with a nested loop in a comprehension. There's no need to repeat the `for` keyword. Like most things in a comprehension, the ordering is backwards from writing things out longhand:

```
for (j, y) in enumerate(y[2:N])
    for (i, x) in enumerate(x[2:N])
        F[i, j] = f(x, y)
    end
end
```

Thus, the *first* loop in the comprehension varies the most quickly.

Warning

History unfolded such that in a matrix, the first dimension is vertically oriented, while for a bivariate function, the first dimension is horizontally oriented. Because MATLAB predates widespread use of graphics-cabable displays, it adopted the convention that in a matrix that represents the discretization of a bivariate function, the first dimension is vertically oriented. Matplotlib, being patterned after MATLAB, adopted the same convention. In modern contexts, this choice can create a lot of confusion, and it's particularly unfortuitous when there are more than two independent variables.

In Julia, the `Plots` package follows the MATLAB convention, while `Makie` more naturally has the first dimension (rows) map to the first variable (x). If you see codes that swap the placement of x and y from what I use, or that have transposed matrices in plotting calls, that's probably what's going on.

Identity matrix

The `I` in line 11 is defined in the `LinearAlgebra` package. Given on its own, `I` represents the identity matrix of whatever size is needed for the context. Here, where the size cannot be inferred, I have given it explicitly. Keep in mind that `I` is an abstract matrix, not a true one; in some situations, you may want to convert it, as in `Matrix(I(n))`.

Kronecker product

The `kron` function, defined in `LinearAlgebra`, computes the Kronecker product of two matrices. My use of row dimension for x and column dimension for y means that `kron(I(N-1), D^2)` corresponds to the second derivative in x, while `kron(D^2, I(N-1))` corresponds to the second derivative in y. Since we treat the dimensions identically here, of course, the distinction doesn't actually matter.

Δ

If being able to use Δ as the name of the discrete Laplacian doesn't convince you to use Unicode in Julia, then I don't know what to tell you. You can get it by typing , then . Better yet, define a text replacement rule or a keyboard mapping for it.

Spy plot

The `spy!` function in line 13 creates a plot in Makie to show thesparsity pattern of a matrix. Note that I supplied `yreversed=true` in the axis creation to make the vertical dimension increase going downward, like row index does.

Timing

The `@time` macro in line 17 reports execution time and memory allocation for the following expression. It's part of base Julia, which is convenient. But it's not as reliably reproducible as `BenchmarkTools` if you need comparative timing information.

Reshaping

We have two inverse types of reshaping in this program, to go between a matrix representing values on a grid and its interpretation as a linear vector. We could do both using `reshape`, but I have used `vec` to go from matrix to vector.

Be advised that reshaping does not copy data on its own, so you can end up with two variables that point to the same data, as in this example:

```
a = ones(4 * 6 * 10)
A = reshape(a, :, 6, 10)    # one dimension size can be left out
a[31] = 0
findfirst(iszero, A)
```

There is a more efficient but clunkier alternative to line 21:

```
vv = reshape(v, N -1, N -1)
copyto!(V, CartesianIndices((2:N, 2:N)), vv, CartesianIndices((1:N -1, 1:N -
1)));
```

Integer division

The `div` function in line 22 performs integer division (quotient without remainder). You can write it in infix form as `3N ⊘ 4`, typing then to get the symbol. To get the remainder, you can use `rem`, and to get both at once, you can use `divrem`.

Surface plot

A surface is drawn in an `Axis3` object. For plots of functions that go roughly the same above and below $z = 0$, I like to use a divergent colormap, like `:redsblues`, and to set the color range to be symmetric about zero, as in `colorrange=(-M, M)`.

> **`rowsize!`**
> If you don't indicate otherwise, each subplot in a grid layout gets the same amount of space. In line 34 I use `rowsize!` to give the first row just 35% of the total height of the figure.

Interpolation in 2-D

I have chosen to use 2-D Chebyshev interpolation to get data on the plotting grid. You might not notice the difference from a low-order method, such as bicubic spline interpolation, on a plot, but the numerical values would be different—and the low-order ones would be much less accurate. Fortunately, it is not hard to turn one-dimensional interpolation into 2-D interpolation on a tensor product grid, as I show in the `interp2dgrid` function from `SpectralMethodsTrefethen`.

Suppose we have an interpolation method F that turns values on a grid $x_1, \ldots, x_m$ into a function f,

$$f(\xi) = F(v_1, \ldots, v_m; \xi), \tag{3}$$

and another method G that does the same for a grid $y_1, \ldots, y_n$,

$$g(\eta) = G(w_1, \ldots, w_n; \eta). \tag{4}$$

Then with a matrix of grid values, $V_{ij} = u(x_i, y_j)$, we can define a 2-D interpolant by a nested application of the 1-D methods:

$$\begin{aligned} u(\xi, \eta) &= G(w_1(\xi), \ldots, w_n(\xi); \eta), \\ w_j(\xi) &= F(V_{1j}, \ldots, V_{mj}; \xi), \quad j = 1, \ldots, n. \end{aligned} \tag{5}$$

Evaluation on a grid of values $\xi_1, \ldots, \xi_p$ and $\eta_1, \ldots, \eta_q$ leads to

$$\begin{aligned} W_{ij} &= w_j(\xi_i) = F(V_{1j}, \ldots, V_{mj}; \xi_i), \quad i = 1, \ldots, p, \\ U_{ij} &= u(\xi_i, \eta_j) = G(W_{i1}, \ldots, W_{in}; \eta_j), \quad j = 1, \ldots, q. \end{aligned} \tag{6}$$

Here is my compact implementation of this idea:

```
function interp2dgrid(V, method1, method2, \xi, \eta)
    W = mapslices(v -> method1(v).( \xi), V; dims=[1])
    return mapslices(w -> method2(w).( \eta), W; dims=[2])
end
```

The first call to `mapslices` applies the first interpolation method to each column of data, evaluating at the given values of ξ, to get the matrix W defined above. The second call applies the second interpolation method to each row of W, evaluating at the given values of η, to get the final matrix U.

0.7.5 Program 17

```
using CairoMakie, Printf, LaTeXStrings
using LinearAlgebra, SpectralMethodsTrefethen
"""
p17 - Helmholtz eq. u_xx + u_yy + kšu = f on [ -1,1] x [ -1,1] (compare p16)
"""
function p17(N = 24, k = 9)
    # Set up spectral grid and tensor product Helmholtz operator:
    D,x = cheb(N)
    y = x
    F = [ exp( -10*((y - 1)^2 + (x - 0.5)^2)) for x in x[2:N], y in y[2:N] ]
    Dš = (D^2)[2:N,2:N]
```

```
    L = kron(I(N -1), Dš) + kron(Dš, I(N -1)) + k^2 * I    # Helmholtz

    # Solve for u, reshape to 2D grid, and plot:
    v = L \ vec(F)
    V = zeros(N+1, N+1)
    V[2:N, 2:N] = reshape(v, N -1, N -1)
    xx = yy = range( -1, 1, 201)
    UU = interp2dgrid(V, chebinterp, chebinterp, xx, yy)

    fig = Figure(size=(600, 800))
    ax1 = Axis3(fig[1, 1]; title="Solution of Helmholtz equation",
      xlabel=L"x", ylabel=L"y", zlabel=L"u(x,y)")
    M = maximum(abs, UU)
    surface!(ax1, xx, yy, UU; colormap=:redsblues, colorrange=( -M, M))
    icen = div(N, 2) + 1
    text!(ax1, -1, 1, M; text="u(0, 0) = $(@sprintf("%0.11f", V[icen, icen]))")
    ax2 = Axis(fig[2, 1]; xlabel=L"x", ylabel=L"y", aspect=1)
    contour!(ax2, xx, yy, UU; levels=range( -M, M, 11), colormap=:redsblues)
    lines!(ax2, [ -1, -1, 1, 1, -1], [ -1, 1, 1, -1, -1], color=:black, linewidth=2)
    hidedecorations!(ax2; label=false, ticks=false)
    return fig
end
```

Nothing new needed here; just relax and enjoy the view.

Output 17

```
p17()
```

0.8 Chapter 8

0.8.1 chebfft

```
DCT(v) = FFTW.r2r(v, FFTW.REDFT00)
DST(v) = FFTW.r2r(v, FFTW.RODFT00)

"""
    chebfft(v)
Differentiate values given at Chebyshev points via the FFT.
"""
function chebfft(v)
    N = length(v) - 1
    (N == 0) && return zero(v)
    x = [cospi(k / N) for k in 0:N]
    v = DCT(v)
    w = similar(v)
    if N > 1
        \xi = 1:N -1
        q = DST( - \xi .* v[2:N])
        w[2:N] = @. -q / (2N * sqrt(1 - x[2:N]^2))
    end
    w[1] = N/2 * v[N+1] + sum(n^2 * v[n+1] for n in 0:N -1) / N
    w[N+1] = ( -1)^(N+1) * N/2 * v[N+1] +
        sum(( -1)^(n+1) * n^2 * v[n+1] for n in 0:N -1) / N
    return w
end
```

I have structured my implementation of `chebfft` to take advantage of the factor of 4 efficiency mentioned in the text. The `FFTW` package gives us access to the discrete cosine and sine transforms, which I give the more convenient names `DCT` and `DST`. The DCT implicitly assumes even symmetry, which avoids the need to create an extended vector of the data. Once we differentiate cosines to get sines, we have an odd function, so the DST is the natural choice for the inverse transform; it expects us to omit the values we know to be zero at the endpoints. Everything else operates much like it does in the MATLAB code.

> **`similar`**
> The `similar` function creates an uninitialized array with the same size and element type as the given array. We have to be careful here not to specify either a real or complex array, since we want the result to have the same type as the input. We could get a valid result using `w = copy(v)`, but that causes an unnecessary copy of the data.

0.8.2 Program 18

```
using CairoMakie, LaTeXStrings, SpectralMethodsTrefethen, ForwardDiff
"p18 - Chebyshev differentiation via FFT (compare p11)"
function p18()
    f(x) = exp(x) * sin(5x)
    fprime(x) = ForwardDiff.derivative(f, x)
    fig = Figure()
    for (i, N) in enumerate([10, 20])
        _, x = cheb(N)
```

```
        v = f.(x)

        Axis(fig[i, 1]; title=latexstring("\$u(x),\\,  N=$N\$"))
        scatter!(fig[i, 1], x, v)
        lines!(fig[i, 1], -1..1, f)

        error = chebfft(v) - fprime.(x)
        Axis(fig[i, 2]; title=latexstring("error in \$u'(x),\\,  N=$N\$"))
        scatter!(fig[i, 2], x, error)
        lines!(fig[i, 2], -1..1, chebinterp(error))
    end
    return fig
end
```

Output 18

```
using SpectralMethodsTrefethen
p18()
```

I made a change compared to Output 11. For the right-hand column, I show the discrete error as markers as usual, but instead of connected them with straight lines, I use `chebinterp` to show the error as a continuous function. I like this here because it dispels the appearance that the discrete errors are local extrema.

0.8.3 Program 19

```
using CairoMakie, LaTeXStrings, FFTW, SpectralMethodsTrefethen
"p19 - 2nd -order wave eq. on Chebyshev grid (compare p6)"
function p19(N=80, tmax=4, \Deltat=8/N^2)
    _, x = cheb(N)
    v = @. exp( -200x^2)
    vold = @. exp( -200 * (x - \Deltat)^2)

    # Time -stepping by leap frog formula:
    tplot = 0.025
    plotgap = round(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)
    data = hcat(v, zeros(N+1, ntime))
    t = \Deltat * (0:ntime)
    for i = 1:ntime
        w = chebfft(chebfft(v))
        w[[1, N+1]] .= 0
        vnew = 2v - vold + \Deltat^2 * w
        data[:, i+1] = vnew
        vold, v = v, vnew
    end

    # Plot results:
    return heatmap(x, t[1:plotgap:end], data[:, 1:plotgap:end];
        colormap=:redsblues, colorrange=( -1, 1),
        axis=(xlabel=L"x", ylabel=L"t"))
end
```

Output 19

```
p19()
```

The reds–blues colormap makes it perfectly clear that the bump inverts whenever it reflects from a boundary. The animation is fun as well:

```
using CairoMakie, Printf, LaTeXStrings
using FFTW, SpectralMethodsTrefethen
"p19anim - 2nd -order wave eq. on Chebyshev grid (compare p6)"
function p19anim(N=80, tmax=4, \Deltat=8/N^2)
    _, x = cheb(N)

    tplot = 0.005
    plotgap = round(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    # Time -stepping by leap frog formula:
    time = Observable(0.0)
    title = @lift latexstring(@sprintf("\$t = %0.2f\$", $time))
    v = Observable(@. exp( -200x^2))
    vold = @. exp( -200 * (x - \Deltat)^2)
    fig = lines(x, v; axis=(; xlabel=L"x", title, limits=( -1, 1, -1, 1)))
    anim = record(fig, "p19anim -$N -$tmax.mp4"; framerate=60) do io
        recordframe!(io)
        for n in 1:ntime
            w = chebfft(chebfft(v[]))
            w[[1, N+1]] .= 0
            vnew = 2v[] - vold + \Deltat^2 * w
            vold = v[]
            v[] = vnew
            time[] = n * \Deltat
            iszero(mod(n, plotgap)) && recordframe!(io)
        end
      end
    return anim
end

p19anim()
```

Compare this solution to Output 6, where we had a variable speed on a periodic domain.

Your eye tells you that this solution is underresolved, but spectral methods are meant to run at coarse resolutions. To smooth out the animation, it would be easy to use the Chebyshev interpolant on a finer plotting grid.

0.8.4 Program 20-anim

```
using CairoMakie, LaTeXStrings, Printf
using FFTW, SpectralMethodsTrefethen
"p20anim - 2nd -order wave eq. in 2D via FFT (compare p19)"
function p20anim(N=24, tmax=1, \Deltat=6/N^2)
    # Grid and initial data:
    x = y = cheb(N)[2]
```

```
    DCT(v) = FFTW.r2r(v, FFTW.REDFT00)
    DST(v) = FFTW.r2r(v, FFTW.RODFT00)
    function fftderiv2(v)
        N = length(v) - 1
        x = [cospi(k / N) for k in 0:N]
        v = DCT(v)
        dQ_d \theta = DST( -(1:N -1) .* v[2:N]) / 2N    # omits n=0, n=N
        dšQ_d \thetaš = DCT( -(0:N).^2 .* v) / 2N
        w = zero(v)    # return zero at boundaries
        # Chain rule for \theta -> x:
        for n in 2:N
            sš = 1 / (1 - x[n]^2)
            s = sqrt(sš)
            w[n] = sš * (dšQ_d \thetaš[n] - x[n] * dQ_d \theta[n -1] * s)
        end
        return w
    end

    V = [exp( -40 * ((x - 0.4)^2 + y^2)) for x in x, y in y]
    Vold = copy(V)
    Vxx = similar(V)
    Vyy = similar(V)

    tplot = 1 / 60
    plotgap = round(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    # Time -stepping by leap frog formula:
    time = Observable(0.0)
    title = @lift latexstring(@sprintf("\$t = %0.2f\$", $time))
    V = Observable(V)
    xx = yy = range( -1, 1, 151)
    VV = @lift interp2dgrid($V, chebinterp, chebinterp, xx, yy)
    fig = Figure()
    ax = Axis(fig[1, 1]; xlabel=L"x", ylabel=L"y", aspect=DataAspect(),
                limits = ( -1, 1, -1, 1), title)
    heatmap!(ax, xx, yy, VV; colorrange=( -0.75, 0.75), colormap=:redsblues)
    anim = record(fig, "p20anim -$N -$tmax.mp4"; framerate=30) do io
        recordframe!(io)
        for n in 1:ntime
            Vxx = mapslices(fftderiv2, V[]; dims=[1])
            Vyy = mapslices(fftderiv2, V[]; dims=[2])
            Vnew = 2V[] - Vold + \Deltat^2 * (Vxx + Vyy)
            time[] = n * \Deltat
            Vold, V[] = V[], Vnew
            iszero(mod(n, plotgap)) && recordframe!(io)
        end
    end
    return anim
end
```

Output 20-anim

```
p20anim(32, 3)
```

I went straight to the animation this time. I chose to break out the FFT differentiation into its own function, `fftderiv2`. It's patterned after the `chebfft` function, but it returns the second derivative—except at the boundaries, where it returns zero. This ensures that the solution remains constant on the boundary during the time stepping. Note that the second derivative in θ is a cosine series again, so the DCT is needed to invert its transform.

This 1-D function is then mapped to slices along each dimension in order to get the second partials needed for the wave equation. Technically, a true 2-D FFT from `FFTW` would be more efficient than repeated applications of 1-D FFTs. But the efficiency difference is unimportant unless N is quite large, and the extra complication to the code is not worthwhile here.

Caution

It's tempting to distill lines 28–29 into the multiple assignment

```
Vxx = Vyy = similar(V)
```

However, this would make both variables aliases of the same array! We need two separate arrays here.

Color range

The color range for the heatmap is set to $[-0.75, 0.75]$. Hence, values greater or less than these values map to the extreme colors in the colormap. While this makes the plot of the initial condition less visually informative, the amplitude of the solution diminishes quickly, and we get more color during most of the animation than we would by using the range $[-1, 1]$.

0.9 Chapter 9

0.9.1 Program 21

```
using CairoMakie, LaTeXStrings, LinearAlgebra, ToeplitzMatrices
"""
p21 - eigenvalues of Mathieu operator -u_xx + 2 q cos(2x) u\n
         (compare p8 and p. 724 of Abramowitz & Stegun)
"""
function p21(N = 42)
    h = 2 \pi / N
    x = h * (1:N)
    c0 = - \pi^2 / 3h^2 - 1 / 6
    col = [c0; [ ( -1)^(j+1) / 2sin(h * j / 2)^2 for j in 1:N -1 ]]
    Dš = Toeplitz(col, col)    # second -order differentiation matrix
    q = 0:.2:15
    data = zeros(11, length(q))
    for (i, q) in enumerate(q)
        \lambda = eigvals( -Dš + 2q * Diagonal(cos.(2x)) )
        data[:, i] = \lambda[1:11]
    end
    fig = Figure(size=(320, 500))
    ax = Axis(fig[1, 1]; xlabel=L"q", ylabel=L" \lambda", yticks= -24:4:32,
        limits=(0, 15, -24, 32))
    series!(ax, q, data[1:2:end, :], color=:atlantic)
    series!(ax, q, data[2:2:end, :], linestyle=:dot, color=:atlantic)
    return fig
end
```

Output 21

```
using SpectralMethodsTrefethen
p21()
```

`series` plot
The `series` function in Makie is a generalization of `lines` that can handle multiple curves at once. They correspond to the rows of the `data` matrix, which are presented in two batches, even and odd, to use different line styles.

0.9.2 Program 22

```
using CairoMakie, LaTeXStrings, Printf
using LinearAlgebra, ToeplitzMatrices, SpecialFunctions, SpectralMethodsTrefethen
"p22 - 5th eigenvector of Airy equation u_xx = \lambda*x*u"
function p22()
    fig = Figure(size=(600, 600))
    N = [12 24; 36 48]
    for (i, N) in pairs(N)
        D, x = cheb(N)
        Dš = (D^2)[2:N, 2:N]
        \lambda, V = eigen(Dš, diagm(x[2:N]))         # generalized ev problem
        fifth = findfirst(>(0), \lambda) + 4
        \lambda, v = \lambda[fifth], V[:, fifth]
        v = [0; v; 0]                            # extend to booundary
```

```
        v = v / v[div(N, 2) + 1] * airyai(0)    # normalize
        u = chebinterp(v)
        lines(fig[i[1], i[2]], -1..1, u;
            axis=(; title=@sprintf("N = %d,  \lambda = %0.10f", N, \lambda)))
    end
    return fig
end
```

Output 22

```
p22()
```

Generalized eigenvalue problem
The `eigen` function can solve the generalized eigenvalue problem $Av = \lambda Bv$. If B is nonsingular, you can rewrite this as an ordinary eigenvalue problem $B^{-1}Av = \lambda v$, but it is not always desirable to do so. However, that is the algorithm used if B is a `Diagonal` matrix, so here we have to use a generic matrix instead to avoid an error due to the singularity of B.

Special functions
The SpecialFunctions package provides a large collection of special functions, such as the gamma function, Bessel functions, elliptic integrals, the error function, the zeta function, and, as used here, the Airy functions.

`findfirst`
The `findfirst` function returns the index of the first element of an array that satisfies a given condition. The result is either `nothing`, if no elements satisfy the condition, or an integer index. I have used the syntax `>(0)` to create an anonymous function that tests whether its argument is greater than zero, shorthand for `x ->x >0`. Finally, we want the 5th index of the result, so I add 4 since they are in sorted order.

0.9.3 Program 23

```
using CairoMakie, Printf, LaTeXStrings
using LinearAlgebra, SpectralMethodsTrefethen
"""
# p23 - eigenvalues of perturbed Laplacian on [ -1,1] x [ -1,1] (compare p16)
"""
function p23(N=16; perturb=false)
    # Set up tensor product Laplacian and compute 4 eigenmodes:
    D, x = cheb(N)
    y = x

    Dš = (D^2)[2:N, 2:N]
    \Delta = -kron(I(N -1), Dš) - kron(Dš, I(N -1))    # Laplacian
    if perturb
        F = [exp(20 * (y - x - 1)) for x in x[2:N], y in y[2:N]]
        \Delta += Diagonal(vec(F))
    end
    \lambda, V = eigen( \Delta)
```

```
    # Reshape them to 2D grid, interpolate to finer grid, and plot:
    xx = yy = range( -1, 1, 101)
    U = zeros(N+1, N+1)
    fig = Figure(size=(600, 600))
    modes = [1 2; 3 4]
    ax = [Axis(fig[i, j]; aspect=1) for i in 1:2, j in 1:2]
    for (i, mode) in pairs(modes)
        U[2:N, 2:N] = reshape(V[:, mode], N -1, N -1)
        UU = interp2dgrid(U, chebinterp, chebinterp, xx, yy)
        UU /= maximum(abs, UU)
        str = @sprintf(" \lambda = %0.12f \pi^2/4", 4 \lambda[mode] / \pi^2)
        contourf!(ax[i], xx, yy, UU; levels= -1.1:0.2:1.1, colormap=:redsblues)
        ax[i].title = str
        hidedecorations!(ax[i], ticks=false)
    end
    return fig
end
```

Output 23a

Both versions of `p23` are so similar that I have put them into a single function with a keyword argument to control the perturbation.

```
p23()
```

Output 23b

```
p23(perturb=true)
```

Compare the outputs to Output 16, which was a solution of a boundary-value problem rather than an eigenvalue problem.

`contourf`
The `contourf` function in Makie is used to create filled contour plots. The result is a kind of hybrid between a heatmap and a contour plot.

Axis title
Rather than adding the title at the time of axis creation, I have added it by assigning to the `.title` property of the axis after the fact.

Minimizing allocations
Now that you are getting comfortable with basic Julia, it's time to look at one of the finer details. I have used `maximum` in many of the programs, including line 28 of Program 23. The outcome of

```
maximum(abs, UU)
```

is the same as

```
maximum(abs.(UU))
```

But they are evaluated differently. The first version applies the `abs` function as a filter while iterating over the elements of the array. The second version creates a new array of the same size as `UU` to hold the absolute values, and then iterates through that array.

We can see the impact of this difference by using the `@benchmark` macro from the BenchmarkTools package. In absolute terms, the difference is small, but in relative terms, the difference is substantial:

```
using BenchmarkTools
A = randn(500, 500)
@benchmark maximum(abs, $A)

@benchmark maximum(abs.($A))
```

Julia lets you achieve many things in multiple ways. Often, it doesn't matter much which you choose. But if a key section of code is taking more time than you like, Julia gives you the tools and fine-grained control to try alternatives.

0.10 Chapter 10

Here's a look at the unstable version of Program 6:

```
using SpectralMethodsTrefethen
p6anim(128, 6, 1.9/128)
```

You can see the same small error early on as in the stable version, but that is not the real issue here. Rather, we see the high-frequency mode in space grow exponentially until it quickly takes over everything.

Here is the unstable version of p20, the 2D wave equation:

```
p20anim(24, 0.5, 6.6/24^2)
```

You should pause the animation and step through frame by frame to see how the error creeps in from the corners of the domain.

0.10.1 Program 25

```
using CairoMakie, Polynomials
"p25 - stability regions for ODE formulas"
function p25()
    fig = Figure(size=(640, 640))
    ax = [Axis(fig[i, j]; aspect=DataAspect()) for i in 1:2, j in 1:2]
    for ax in ax
        hlines!(ax, [0]; color=:gray)
        vlines!(ax, [0]; color=:gray)
    end

    z = [cispi(x) for x in (0:200) / 100]
    # AdamsBashforth, orders 1 -3:
    \rhoz = @. z - 1
    for \sigma in Polynomial.([[1], [ -1, 3] / 2, [5, -16, 23] / 12])
        w = @. \rhoz / \sigma(z)
        lines!(ax[1, 1], reim(w)...)
        @. \rhoz *= z
    end
    ax[1, 1].title = "AdamsBashforth"

    # AdamsMoulton, orders 3 -6:
    \rhoz = @. z^2 - z
    for \sigma in Polynomial.([
            [ -1, 8, 5] / 12,
            [1, -5, 19, 9] / 24,
            [ -19, 106, -264, 646, 251] / 720,
            [27, -173, 482, -798, 1427, 475] / 1440
        ])
        w = @. \rhoz / \sigma(z)
        lines!(ax[1, 2], reim(w)...)
        @. \rhoz *= z
    end
    ax[1, 2].title = "AdamsMoulton"
```

```
    # Backward differentiation, orders 1 -6:
    d = @. 1 - 1 / z
    r = zero(d)
    for n in 1:6
        @. r += (d ^ n) / n
        lines!(ax[2, 1], reim(r)...)
    end
    ax[2, 1].title = "backward differentiation"

    # Runge -Kutta, orders 1 -4:
    T = [Polynomial(1 ./ factorial.(0:n)) for n in 0:4]
    w = zero(z)
    for n in 1:4
        for i in 1:length(z) -1
            w[i+1] = w[i] - (T[n+1](w[i]) - z[i+1]^n) / T[n](w[i])
        end
        lines!(ax[2, 2], reim(w)...)
    end
    ax[2, 2].title = "RungeKutta"
    return fig

end
```

Output 25

```
using SpectralMethodsTrefethen
p25()
```

I rearranged the computations a bit. For the Adams methods, I've referred to the standard characteristic polynomials ρ and σ, and for the Runge–Kutta methods, I've drawn more attention to the connection to the Maclaurin series of the exponential function.

> **`reim`**
> I've used `reim(w)...` as a slight shorthand for `real(w), imag(w)`. The `reim` function returns a tuple of the real and imaginary parts of its argument, and the splatting operator unpacks the tuple into separate arguments.
>
> A subtle point: the real part of a vector is a vector, so it's okay to use `real(w)`, though the broadcasted version `real.(w)` would also work. The same is true for `imag`. However, broadcasting `reim` returns a vector of tuples rather than a tuple of vectors and would not splat in a useful way here. In Makie, you could use `Point2.(reim.(w))` to get an equivalent plot.

0.10.2 Program 26

```
using CairoMakie, LaTeXStrings
using LinearAlgebra, SpectralMethodsTrefethen
"p26 - eigenvalues of 2nd -order Chebyshev diff. matrix"
function p26(N=60)
    D, x = cheb(N)
    Dš = (D^2)[2:N, 2:N]
    \lambda, V = eigen( -Dš)

    fig = Figure(size=(600, 800))
    # Plot eigenvalues:
```

```
    M = round(maximum( \lambda) / N^4, sigdigits=5)
    str = "\$N = $N\$,     max \$\\!|\\lambda| = $M N^4\$"
    ax1 = Axis(fig[1, 1]; title=latexstring(str),
        xscale=log2, xlabel="mode", yscale=log10, ylabel="eigenvalue")
    scatter!(ax1, \lambda)
    vlines!(ax1, [2N / \pi], linestyle=:dash, color=:red)
    text!(ax1, 1.9N / \pi, 24; text=L"2\pi / N", align=(:right, :baseline))

    # Plot eigenmodes N/4 (physical) and N (nonphysical):
    vN4 = [0; V[:, div(N, 4) - 1]; 0]
    uN4 = chebinterp(vN4)
    ax2 = Axis(fig[2, 1]; xlabel=L"x", ylabel=L"u(x)", title=latexstring("eigenmode \$N/4\$"))
    scatter!(ax2, x, vN4)
    lines!(ax2, -1..1, uN4)

    vN = V[:, N -1];
    yticks = ([ -1, -1e -2, -1e -4, 1e -4, 1e -2, 1],
            ["1", "10š", "10", "10", "10š", "1"])
    ax3 = Axis(fig[3, 1]; xlabel=L"x", limits=( -1.05, 1.05, -1.25, 1.25),
        yscale=Makie.AsinhScale(1e -5), yticks, ylabel=L"v_n",
        title=latexstring("eigenmode \$N\$ (asinh scale)"))
    scatterlines!(ax3, x[2:N], vN; linestyle=:dot)

    rowsize!(fig.layout, 1, Relative(0.4))
    return fig
end
```

Output 26

```
p26()
```

asinh scaling

The third plot is supposed to show values that oscillate around zero, but at magnitudes that are more logarithmic than they are linear. Makie offers more than just linear and log scales, and here I selected an inverse sinh scale:

$$\tilde{y} = \frac{\operatorname{asinh}(y/a)}{\operatorname{asinh}(1/a)}, \tag{7}$$

where a is a parameter that controls the transition between linear and logarithmic behavior. The `Makie.AsinhScale` constructor takes this parameter as an argument.

0.10.3 Program 27

```
using CairoMakie, LaTeXStrings, FFTW
"""
p27 - Solve KdV eq. u_t + u u_x + u_xxx = 0 on [ - \pi, \pi] by
        FFT with integrating factor v = exp( -ikşt) * u.
"""
function p27(N=256, tmax=0.006, \Deltat=0.4/N^2)
    @assert iseven(N) "Must choose an even value of N"
    # Set up grid and two -soliton initial data:
    x = (2 \pi / N) * ( -N/2:N/2 -1)
    soliton(x, a) = 3a^2 * sech(0.5 * a * x)^2
```

```
    u = @. soliton(x + 2, 25) + soliton(x + 1, 16)

    # Set up time stepping:
    tplot = tmax / 80
    plotgap = round(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    # Find integrating factor and solve PDE:
    k = [0:N/2 -1; 0]
    ikş = 1im * k .^ 3
    g = -0.5im * \Deltat * k
    E = exp.( \Deltat * ikş / 2)
    Eš = E .^ 2
    u = rfft(u)
    udata = u
    tdata = [0.0]
    for n = 1:ntime
        t = n * \Deltat
        a = g .* rfft(irfft(û, N) .^ 2)
        b = g .* rfft(irfft(E .* (û + a / 2), N) .^ 2)    # 4th -order...
        c = g .* rfft(irfft(E .* û + b / 2, N) .^ 2)      # ...RungeKutta
        d = g .* rfft(irfft(Eš .* û + E .* c, N) .^ 2)
        û = @. Eš * û + (Eš * a + 2 * E * (b + c) + d) / 6
        if iszero(rem(n, plotgap))
            u = irfft(û, N)
            udata = [udata u]
            tdata = [tdata; t]
        end
    end
    return heatmap(x, tdata, udata; colormap=:Blues, interpolate=true,
        axis=(xlabel=L"x", ylabel=L"t"))
end
```

Output 27

```
p27()
```

The heatmap in this case shows clearly the way the humps emerge from their interaction shifted from their original characteristic paths. That's a nonlinear effect in KdV.

Real FFT

Because we use `rfft` and `irfft` for the Fourier transforms, we never have to explictly take the real part of the results, and we enjoy a factor of 2 speedup over a generic FFT.

Interpolated heatmap

I used `interpolate=true` in the `heatmap` call to get a smoother-looking plot. I guess that's a bit hypocritical given my earlier stances on keeping spectral accuracy, but interpolating faithfully to 4th order in time is not easy here.

Simple fixed-time-step implementations of methods for IVPs are great to learn from, but they should not be used when solving challenging problems. Julia has the best collection of IVP solvers in the world, in the DifferentialEquations package. Most of its sovlers automatically provide

adaptive step size and interpolation in time. The package even offers exponential Runge–Kutta methods ready to use.

0.10.4 Program 27-anim

```
using CairoMakie, LaTeXStrings, Printf, FFTW
"""
p27anim - Solve KdV eq. u_t + u u_x + u_xxx = 0 on [ - \pi, \pi] by
        FFT with integrating factor v = exp( -ikşt) * u.
"""
function p27anim(N=256, tmax=0.006, \Deltat=0.4/N^2)
    @assert iseven(N) "Must choose an even value of N"
    # Set up grid and two -soliton initial data:
    x = (2 \pi / N) * ( -N/2:N/2 -1)
    soliton(x, a) = 3a^2 * sech(0.5 * a * x)^2
    u = @. soliton(x + 2, 25) + soliton(x + 1, 16)

    # Set up time stepping:
    tplot = tmax / 150
    plotgap = ceil(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    # Find integrating factor and solve PDE:
    k = [0:N/2 -1; 0]
    ikş = 1im * k .^ 3
    g = -0.5im * \Deltat * k
    E = exp.( \Deltat * ikş / 2)
    Eš = E .^ 2
    time = Observable(0.0)
    title = @lift @sprintf("t = %0.2e", $time)
    u = rfft(u)
    u = Observable(u)
    fig = lines(x, u;
        axis=(; xlabel=L"x", xticks=MultiplesTicks(5, \pi, " \pi"), title))
    anim = record(fig, "p27anim -$N -$(1000tmax).mp4"; framerate=30) do io
        recordframe!(io)
        for n in 1:ntime
            a = g .* rfft(irfft(û, N) .^ 2)
            b = g .* rfft(irfft(E .* (û + a / 2), N) .^ 2)    # 4th -order...
            c = g .* rfft(irfft(E .* û + b / 2, N) .^ 2)      # ...RungeKutta
            d = g .* rfft(irfft(Eš .* û + E .* c, N) .^ 2)
            û = @. Eš * û + (Eš * a + 2 * E * (b + c) + d) / 6
            if iszero(rem(n, plotgap))
                u[] = irfft(û, N)
                time[] = n * \Deltat
                recordframe!(io)
            end
        end
    end
    return anim
end
```

Output 27-anim

```
p27anim()
```

0.11 Chapter 11

0.11.1 Program 28

```
using CairoMakie, Printf
using ToeplitzMatrices, LinearAlgebra, SpectralMethodsTrefethen
"p28 - eigenmodes of Laplacian on the disk (compare p22)"
function p28(N=25, M=20; version=:a)
    @assert isodd(N) && iseven(M) "Must choose odd N and even M"
    # r coordinate, ranging from -1 to 1 (N must be odd):
    N2 = div(N -1, 2)
    Dr, r = cheb(N)
    Dšr = Dr^2
    D = Dšr[2:N2+1, 2:N2+1]
    D = Dšr[2:N2+1, N: -1:N2+2]
    E = Dr[2:N2+1, 2:N2+1]
    E = Dr[2:N2+1, N: -1:N2+2]

    # \theta coordinate, ranging from 0 to 2 \pi (M must be even):
    d \theta = 2 \pi / M
    \theta = d \theta * (1:M)
    M2 = div(M, 2)
    c0 = - \pi^2 / 3d \theta^2 - 1 / 6
    col = [0.5 * ( -1)^(k + 1) / sin(k * d \theta / 2)^2 for k in 1:M -1]
    Dš \theta = Toeplitz([c0; col], [c0; col])

    # Laplacian in polar coordinates:
    R = Diagonal(1 ./ r[2:N2+1])
    J = [zeros(M2, M2) I; I zeros(M2, M2)]
    \Delta = kron(D + R * E, I(M)) + kron(D + R * E, J) + kron(R^2, Dš \theta)

    # Compute eigenmodes:
    \lambda, W = eigen( - \Delta, sortby=abs)
    \lambda = sqrt.(real( \lambda / \lambda[1]))

    function getmode(W, k, \theta, r)
        V = reshape(real(W[:, k]), M, N2)
        V = [zeros(M) V / maximum(abs, V)]                # BC and normalize
        V = [V V[(@. mod1(M2 + (1:M), M)), N2+1: -1:1]]    # extend to r < 0
        return interp2dgrid(V, fourinterp, chebinterp, \theta, r)
    end

    # Plot eigenmodes:
    r, \theta = range( -1, 1, 71), range(0, \pi, 111)
    X = [r * cos( \theta) for \theta in \theta, r in r]
    Y = [r * sin( \theta) for \theta in \theta, r in r]
    zcirc = cispi.(( -200:200) / 200)
    fig = Figure(size=(640, 640))
    if version == :a
        modes = [1 3; 6 10]
        for (i, mode) in pairs(modes)
            str = @sprintf("Mode %d: \lambda = %0.11f", mode, \lambda[mode])
            U = getmode(W, mode, \theta, r)
            ax = Axis3(fig[i[1], i[2]]; title=str, aspect=:data)
```

```
            surface!(ax, X, Y, 1.5 .+ U;
                colormap=:redsblues, colorrange=(0.5, 2.5))
            contour!(ax, X, Y, U; levels=[0], color=:darkblue, linewidth=1)
            lines!(ax, reim(zcirc)...; color=:black, linewidth=2)
            set_lights!(ax, [])
            set_ambient_light!(ax, RGBf(0.85, 0.85, 0.85))
            hidedecorations!(ax)
            hidespines!(ax)
        end
    elseif version == :b
        modes = reshape(1:25, 5, 5)'
        for (i, mode) in pairs(modes)
            str = @sprintf("%0.4f", \lambda[mode])
            U = getmode(W, mode, \theta, r)
            ax = Axis(fig[i[1], i[2]]; title=str, titlefont=:regular, aspect=DataAspect())
            contour!(ax, X, Y, U; levels=[0], color=:darkblue, linewidth=1)
            lines!(ax, reim(zcirc)...; color=:black, linewidth=2)
            hidedecorations!(ax)
            hidespines!(ax)
        end
    end
    return fig
end
```

Output 28

```
using SpectralMethodsTrefethen
p28()
```

I found the outputs for this chapter in *SMiM* to be a bit coarse-looking, so I have interpolated the results onto a finer polar grid. Now version b is picture-perfect:

```
p28(version=:b)
```

Compare this result to the modes on a square domain.

> **`mod1`**
> The most noteworthy bit of the program is in the `getmode` subfunction. To set up for interpolation on the disk using our standard functions, we need to restore the compact polar representation to all four regions depicted in Figure 11.3 of the text—specifically, to recreate the data for $r < 0$. The data matrix `V` has θ along M rows and r along $(N-1)/2$ columns. To find the data for extending a row, we have to look in the row that is $M/2$ rows away, where we treat the row index circularly.
>
> The `mod1` function is perfect for working with periodic 1-indexed arrays. It computes the index modulo M, but with the result in $0 < m \le M$ instead of $0 \le m < M$. Of course, we also have to reverse the order of the columns as we append to each row. Once that is taken care of, the `interp2dgrid` function doesn't care that our grid is polar—it just sees a rectangular grid with our Fourier setup in the row dimension and Chebyshev in the column dimension.

0.11.2 Program 29

```
using CairoMakie, LaTeXStrings
using ToeplitzMatrices, SpectralMethodsTrefethen
"p29 - solve Poisson equation on the unit disk (compare p16 and p28)"
```

```
function p29(N=31, M=40)
    @assert isodd(N) && iseven(M) "Must choose odd N and even M"
    # Set up r and \theta:
    N2, M2 = div(N - 1, 2), div(M, 2)
    Dr, r = cheb(N)
    Dšr = Dr^2
    D = Dšr[2:N2+1, 2:N2+1]
    D = Dšr[2:N2+1, N: -1:N2+2]
    E = Dr[2:N2+1, 2:N2+1]
    E = Dr[2:N2+1, N: -1:N2+2]
    d \theta = 2 \pi / M
    \theta = d \theta * (1:M)
    c0 = - \pi^2 / 3d \theta^2 - 1 / 6
    col = [0.5 * ( -1)^(k + 1) / sin(k * d \theta / 2)^2 for k in 1:M -1]
    Dš \theta = Toeplitz([c0; col], [c0; col])

    # Laplacian in polar coordinates:
    R = Diagonal(1 ./ r[2:N2+1])
    J = [zeros(M2, M2) I; I zeros(M2, M2)]
    \Delta = kron(D + R * E, I(M)) + kron(D + R * E, J) + kron(R^2, Dš \theta)

    # Right -hand side and solution for u:
    F = [ -r^2 * sin( \theta / 2)^4 + sin(6 \theta) * cos( \theta / 2)^2 for \theta in \theta,
    v = \Delta \ vec(F)

    # Reshape results, extend to r < 0, and interpolate for the plot:
    V = reshape(v, M, N2)
    V = [zeros(M) V]                                # boundary values at r = 1
    V = [V V[(@. mod1(M2 + (1:M), M)), N2+1: -1:1]]    # extend to r  [ -1, 0)
    r, \theta = range( -1, 1, 71), range(0, \pi, 111)
    U = interp2dgrid(V, fourinterp, chebinterp, \theta, r)
    X = [r * cos( \theta) for \theta in \theta, r in r]
    Y = [r * sin( \theta) for \theta in \theta, r in r]
    fig, ax, plt = contourf(X, Y, U; levels=20, colormap=:amp,
        axis=(; xlabel=L"x", ylabel=L"y", aspect=1))
    Colorbar(fig[1, 2], plt)
    return fig
end
```

Output 29

```
p29()
```

All the heavy lifting was done already for Program 28. Note that here, as there, I chose to plot using the part of the data corresponding to $r \in [-1, 1]$ and $\theta \in [0, \pi]$. This fixes the hole you can see in the mesh of Output 29 in *SMiM*.

> **`Colorbar`**
> The `Colorbar` function from Makie is used to add a colorbar to any part of the subplot grid you would like. Given the filled contour object we plotted, it uses the same colormap and color range.

0.12 Chapter 12

0.12.1 Program 30

```
using CairoMakie, LaTeXStrings
using SpectralMethodsTrefethen, SpecialFunctions
"""
p30 - spectral integration, ODE style (compare p12)\n
   use ‘chebweights‘, ‘clencurt‘, or ‘gauss‘ as the argument
"""
function p30(quadrule=chebweights)
    # Computation: various values of N, four functions:
    Nmax = 50
    funs = [x -> abs(x)^3         x -> exp( -x^( -2));
            x -> 1 / (1 + x^2)    x -> x^10]
    ints = [1/2 2*(exp( -1) + sqrt( \pi) * (erf(1) - 1));  \pi/2 2/11]
    E = zeros(2, 2, Nmax)
    for N = 1:Nmax
        x, w = quadrule(N)
        for (i, f) in pairs(funs)
            E[i, N] = abs(dot(w, f.(x)) - ints[i])
        end
    end

    # Plot results:
    fig = Figure()
    titles = [L"|x|^3"  L"\exp( -x^2)"; L"1/(1+x^2)"  L"x^{10}"]
    ax = [Axis(fig[i, j]; title=titles[i, j], yscale=log10) for i in 1:2, j in 1:2]
    linkaxes!(ax...)
    @. E[iszero(E)] = NaN
    for (i, ax) in pairs(ax)
        scatterlines!(ax, 1:Nmax, E[i, :])
    end
    ax[1, 1].ylabel = ax[2, 1].ylabel = "error"
    ax[2, 1].xlabel = ax[2, 2].xlabel = L"N"
    return fig
end

function chebweights(N)
    i = 1:N
    D, x = cheb(N)
    x = x[i]
    w = inv(D[i, i])[1, :]
    return x, w
end
```

By default, Program 30 computes quadrature weights by inverting a Chebyshev differentiation matrix, as in the text. I put that step into a function called `chebweights`, which is replaceable by the other two quadrature formulas in the chapter.

Output 30

```
using SpectralMethodsTrefethen
p30()
```

NaN values

In line 26 of Program 30, I have replaced all the zero error values with `NaN` (Not a Number). That's what causes the gaps you can see in the lower two plots of Output 30a.

A `NaN` value can arise as the result of an undefined or indeterminate operation. Traditionally, it has also been used to represent missing values, and it remains convenient to use it that way in plots. But Julia also offers a separate `missing` value for that purpose, which is more appropriate for working with other kinds of data.

This brings up an amusing bit of Julia:

```
NaN isa Number
```

`NaN` is a special floating-point value, and all floating-point values are of type `Number`, so the assertion is true.

Caution

Numerical operations and comparisons involving `NaN` always result in `NaN`. Most notoriously,

```
NaN == NaN
```

To test for a `NaN` value, you should use the `isnan` function.

0.12.2 clencurt

```
"""
    clencurt(N)
Nodes and weights for ClenshawCurtis quadrature.
"""
function clencurt(N, T::Type{<:AbstractFloat}=Float64)
    q = [T(i) / N for i in 0:N]
    x = cospi.(q)
    w = zeros(T, N+1)
    v = ones(T, N -1)
    if iseven(N)
        w[1] = w[N+1] = one(T) / (N^2 - 1)
        for k in 1:div(N, 2) -1
            @. v -= 2cospi(2k * q[2:N]) / (4k^2 - 1)
        end
        @. v -= cospi(N * q[2:N]) / (N^2 - 1)
    else
        w[1] = w[N+1] = one(T) / N^2
        for k in 1:div(N -1, 2)
            @. v -= 2cospi(2k * q[2:N]) / (4k^2 - 1)
        end
    end
    w[2:N] = 2v / N
    return x, w
end
```

I've had some fun here by extending `clencurt` to work with floating-point types other than double precision, which is called `Float64` in Julia. Base Julia has some less-precise types, such as `Float32`, and some more-precise types, such as `BigFloat`, which is a variable-precision type. There are also

add-on packages such as `DoubleFloats` and `MultiFloats` that can give higher fixed precision with greater speed than `BigFloat`.

All that Clenshaw–Curtis requires is basic arithmetic and the cosine function, so it is easy to generalize. One has to be mindful that some operations, such as the ratio of integers, default to `Float64`, which will limit the precision of all future operations. Thus, to define `q` in line 6, I explicitly cast the numerator to have type `T`. Since the denominator is an integer, it will be implicitly converted to `T` as well before the division is performed.

Now we can integrate as accurately as we like:

```
T = BigFloat
setprecision(100)    # 100 bits of precision, about 30 decimal digits
println("first BigFloat after 1 is ", nextfloat(one(T)))
x, w = clencurt(40, T)
@show sum(w * exp(2x) for (x, w) in zip(x, w)) - sinh(T(2));
```

Output 30b

```
p30(clencurt)
```

0.12.3 gauss

```
"""
    gauss(N)
Nodes and weights for Gauss quadrature.
"""
function gauss(N)
    \beta = [0.5 / sqrt(1 - 1 / 4i^2) for i in 1:N -1]
    T = SymTridiagonal(zeros(N), \beta)
    x, V = eigen(T)
    return x, 2V[1, :] .^ 2
end
```

If we want to extend `gauss` to work in higher precision, we need an eigensolver that works with that type. Those exist, but the relative cost is much higher than for Clenshaw–Curtis.

> **`SymTridiagonal`**
> The `SymTridiagonal` type is a special type for symmetric tridiagonal matrices. It is used here because the matrix in the Golub–Welsch algorithm has those properties, and the eigensolver for that case is a lot more efficient than the general-purpose one.

Output 30c

```
p30(gauss)
```

0.12.4 Program 31

```
using CairoMakie, SpectralMethodsTrefethen, SpecialFunctions
"p31 - gamma function via complex integral, trapezoid rule"
function p31(N = 70, c = -11, r = 16)
    # points on circle of integration:
    t = [c + r * cispi(q) for q in 2 * (0.5:N -0.5) / N]
    # trapezoid integration:
    gaminv(z) = sum(exp(t) * t^( -z) * (t - c) for t in t) / N
```

```
    x, y = range( -3.5, 4, 121), range( -2.5, 2.5, 121)
    \Gamma = [1 / gaminv(complex(x, y)) for x in x, y in y]
    fig = Figure(size=(700, 500))
    ax = Axis3(fig[1, 1]; xlabel="Re(z)", ylabel="Im(z)", zlabel="| \Gamma(z)|")
    G = @. min(6, abs( \Gamma))
    plt = surface!(ax, x, y, G; colormap=:lajolla, colorscale=log2, colorrange=(1e -
2, 6))
    Colorbar(fig[1, 2], plt)
    return fig
end
```

Output 31

```
p31()
```

0.13 Chapter 13

0.13.1 Program 32

```
using CairoMakie, Printf, SpectralMethodsTrefethen
"p32 - solve u_xx = exp(4x), u( -1) = 0, u(1) = 1 (compare p13)"
function p32(N = 16)
    D, x = cheb(N)
    Dš = (D^2)[2:N, 2:N]                    # boundary conditions
    f = [exp(4x) for x in x[2:N]]
    v = Dš \ f                            # Poisson eq. solved here
    v = [0; v; 0] + (x .+ 1) / 2
    fig, ax, _ = scatter(x, v)
    u = chebinterp(v)
    lines!(ax, -1..1, u)
    exact(x) = (exp(4 * x) - sinh(4) * x - cosh(4)) / 16 + (x + 1) / 2
    maxerr = maximum(abs(u(x) - exact(x)) for x in range( -1, 1, 801))
    ax.title = @sprintf("max err = %0.4e", maxerr)
    return fig
end
```

Output 32

```
using SpectralMethodsTrefethen
p32()
```

Compare to Output 13, which is the same ODE with homogeneous Dirichlet conditions.

There are alternatives to the mathematical trick used here. One is the approach in the upcoming Program 33. Another is to modify the "boundary deletion" strategy we have used throughout the book.

```
using CairoMakie, Printf, SpectralMethodsTrefethen
N = 16
bc = [1, 0]
D, x = cheb(N)
v = zeros(N+1)
idxb = [1, N+1]    # boundary indices
v[idxb] = bc
Dš = D^2
f = (@. exp(4x[2:N])) - Dš[2:N, idxb] * bc    # modify RHS for the BCs
 \Delta = Dš[2:N, 2:N]    # delete boundary rows and columns
v[2:N] = \Delta \ f      # solve for interior values
u = chebinterp(v)
exact(x) = (exp(4 * x) - sinh(4) * x - cosh(4)) / 16 + (x + 1) / 2
maxerr = maximum(abs(u(x) - exact(x)) for x in range( -1, 1, 801))
println("max error is now: ", maxerr)
```

0.13.2 Program 33

```
using CairoMakie, Printf, SpectralMethodsTrefethen
"p33 - solve linear BVP u_xx = exp(4x), u( -1) = u(1) = 0"
function p33(N=16)
    D, x = cheb(N)
    A = D^2
    A[N+1, :] = D[N+1, :]            # Neumann condition at x = -1
```

```
    A[1, :] = I(N+1)[1, :]           # Dirichlet condition at x = 1
    f = [exp(4x) for x in x[2:N]]
    v = A \ [0; f; 0]
    fig, ax, _ = scatter(x, v)
    u = chebinterp(v)
    lines!(ax, -1..1, u)
    exact(x) = (exp(4x) - 4 * exp( -4) * (x - 1) - exp(4)) / 16
    maxerr = maximum(abs(u(x) - exact(x)) for x in range( -1, 1, 801))
    ax.title = @sprintf("max err = %0.4e", maxerr)
    return fig
end
```

Output 33

```
p33()
```

0.13.3 Program 34anim

```
using CairoMakie, Printf, SpectralMethodsTrefethen
"""
p34anim - AllenCahn eq. u_t =  u_xx + u - uş, u( -1) = -1, u(1) = 1\n
         (compare p6 and p32)
"""
function p34anim(N=20, =0.01, tmax=1/)
    # Differentiation matrix and initial data:
    D, x = cheb(N)
    Dš = D^2                 # use full -size matrix
    Dš[[1, N+1], :] .= 0     # for convenience
    v = @. 0.53x + 0.47sinpi( -1.5x)

    # Solve PDE by Euler formula and plot results:
    \Deltat = min(0.01, 20 / (N^4 * ))
    tplot = 0.2
    plotgap = ceil(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    time = Observable(0.0)
    v = Observable(v)
    u = @lift chebinterp($v)
    str = @lift @sprintf("t = %0.1f", $time)
    fig, _ = lines( -1..1, u,
        axis=(; xlabel=L"x", ylabel=L"u(x,t)", title=str) )
    anim = record(fig, "p34anim.mp4"; framerate=60) do io
        recordframe!(io)
        for n in 1:ntime
            vnew = v[]
            vnew += \Deltat * ( * Dš * (vnew - x) + vnew - vnew .^ 3)    # Euler
            v[] = vnew
            time[] = n * \Deltat
            iszero(mod(n, plotgap)) && recordframe!(io)
        end
    end
    return anim
```

```
end
```

Output 34anim

```
p34anim()
```

0.13.4 Program 35anim

```
using CairoMakie, Printf, SpectralMethodsTrefethen
"""
p35anim - Allen -Cahn eq. as in p34, but with boundary condition
              imposed explicitly ("method (II)") w/animated solution
"""
function p35anim(N=20, =0.01, tmax=1/2)
    # Differentiation matrix and initial data:
    D, x = cheb(N)
    Dš = D^2     # use full -size matrix
    v = @. 0.53 * x + 0.47 * sin( -1.5 \pi * x)

    # Solve PDE by Euler formula and plot results:
    \Deltat = min(0.01, 50 / (N^4 * ))
    tplot = 0.2
    plotgap = ceil(Int, tplot / \Deltat)
    \Deltat = tplot / plotgap
    ntime = round(Int, tmax / \Deltat)

    time = Observable(0.0)
    v = Observable(v)
    u = @lift chebinterp($v)
    str = @lift @sprintf("t = %0.1f", $time)
    fig, ax, _ = lines( -1..1, u,
        axis=(; xlabel=L"x", ylabel=L"u(x,t)", title=str, limits=( -1, 1, -1, 2)))
    anim = record(fig, "p35anim.mp4"; framerate=15) do io
        recordframe!(io)
        for n in 1:ntime
            vnew = v[]
            vnew += \Deltat * ( * Dš * (vnew - x) + vnew - vnew .^ 3)    # Euler
            vnew[1], vnew[end] = 1 + sin((n -1) * tplot / 5)^2, -1    # BC
            v[] = vnew
            time[] = n * \Deltat
            iszero(mod(n, plotgap)) && recordframe!(io)
        end
    end
    return anim
end
```

Output 35anim

```
p35anim()
```

0.13.5 Program 36

```
using CairoMakie, Printf, LaTeXStrings
using SpectralMethodsTrefethen, LinearAlgebra
"p36 - Laplace eq. on [ -1,1] x [ -1,1] with nonzero BCs"
```

```
function p36(N=24)
    # Set up grid and 2D Laplacian, boundary points included:
    D, x = cheb(N)
    y = x
    Dš = D^2
    \Delta = kron(I(N+1), Dš) + kron(Dš, I(N+1))

    # Impose boundary conditions by replacing appropriate rows of L:
    isboundary(x, y) = (abs(x) == 1 || abs(y) == 1)
    idxb = vec([isboundary(x, y) for x in x, y in y])       # boundary pts
    \Delta[idxb, :] .= 0
    \Delta[idxb, idxb] .= I(4N)
    rhs = zeros((N+1)^2)
    g(x, y) = (y == 1) && (x < 0) ? sinpi(x)^4 : (x == 1) ? 0.2sinpi(3y) : 0
    rhs[idxb] .= [g(x, y) for x in x, y in y if isboundary(x, y)]

    # Solve Laplace equation, reshape to 2D, and plot:
    v = \Delta \ rhs
    V = reshape(v, N+1, N+1)
    x = y = range( -1, 1, 121)
    U    = interp2dgrid(V, chebinterp, chebinterp, x, y)
    u0 = only(interp2dgrid(V, chebinterp, chebinterp, [0], [0]))
    title = latexstring(@sprintf("u(0, 0) = %0.10f", u0))
    fig = Figure()
    ax = Axis3(fig[1, 1]; title, xlabel=L"x", ylabel=L"y", zlabel=L"u(x,y)")
    surface!(ax, x, y, U)
    return fig
end
```

Output 36

```
p36()
```

0.13.6 Program 37anim

```
using CairoMakie, Printf, LaTeXStrings
using SpectralMethodsTrefethen, LinearAlgebra, ToeplitzMatrices
"p37 - 2D \"wave tank\" with Neumann BCs for |y| = 1"
function p37anim(Nx=50, Ny=15, tmax=4)
    # x variable in [ -A, A], Fourier:
    A = 3
    \Delta \theta = 2 \pi / Nx
    \theta = \Delta \theta * (1:Nx)
    c0 = - \pi^2 / 3 \Delta \theta^2 - 1 / 6
    col = [0.5 * ( -1)^(k + 1) / sin(k * \Delta \theta / 2)^2 for k in 1:Nx -1]
    Dš \theta = Toeplitz([c0; col], [c0; col])
    x = \theta -> A * \theta / \pi - A
    Dšx = ( \pi / A)^2 * Dš \theta

    # y variable in [ -1, 1], Chebyshev:
    Dy, y = cheb(Ny)
    Dšy = Dy^2
    idxb, idxi = [1, Ny+1], 2:Ny
    BC = -Dy[idxb, idxb] \ Dy[idxb, idxi]
```

```
    # Initial data:
    V = [exp( -8 * ((x + 1.5)^2 + y^2)) for x in x.( \theta), y in y]
    \Deltat = 1 / (Nx + Ny^2)
    Vold = [exp( -8 * ((x + \Deltat + 1.5)^2 + y^2)) for x in x.( \theta), y in y]

    # Time -stepping by leap frog formula:
    ntime = ceil(Int, tmax / \Deltat)
    \Deltat = tmax / ntime
    time = Observable(0.0)
    V = Observable(V)
    \theta \theta = range(0, 2 \pi, 111)
    yy = range( -1, 1, 51)
    U = @lift interp2dgrid($V, fourinterp, chebinterp, \theta \theta, yy)
    title = @lift latexstring(@sprintf("t = %0.2f", $time))
    fig = Figure()
    ax = Axis(fig[1, 1]; title, aspect=DataAspect(), xlabel=L"x", ylabel=L"y")
    heatmap!(ax, x.( \theta \theta), yy, U; colormap=:redsblues, colorrange=( -
0.75, 0.75), interpolate=true)
    anim = record(fig, "p37anim.mp4"; framerate=60) do io
        recordframe!(io)
        for i in 1:ntime
            Vnew = 2V[] - Vold + \Deltat^2 * (Dšx * V[] + V[] * Dšy')
            Vnew[:, idxb] .= Vnew[:, idxi] * BC'       # Neumann BCs for |y|=1
            time[] = i * \Deltat
            Vold, V[] = V[], Vnew
            recordframe!(io)
        end
    end
    return anim
end
```

Output 37anim

```
p37anim()
```

0.14 Chapter 14

0.14.1 Program 38

```
using CairoMakie, Printf
using LinearAlgebra, SpectralMethodsTrefethen, Polynomials
"p38 - solve u = exp(x), u( -1) = u(1) = u( -1) = u(1) = 0 (compare p13)"
function p38(N=15)
    # Construct discrete biharmonic operator:
    D, x = cheb(N)
    s = [1 / (1 - x^2) for x in x[2:N]]
    S = Diagonal([0; s; 0])
    \Deltaš = (Diagonal(@. 1 - x ^ 2) * D^4 - Diagonal(8x) * D^3 - 12D^2) * S
    \Deltaš = \Deltaš[2:N, 2:N]

    # Solve boundary -value problem and plot result:
    rhs = @. exp(x[2:N])
    v = \Deltaš \ rhs
    v = [0; v; 0]
    fig, ax, _ = scatter(x, v)
    q = chebinterp(S * v)
    u(x) = (1 - x^2) * q(x)
    lines!(ax, -1..1, u)

    # Determine exact solution and print maximum error:
    c = [1 -1 1 -1; 0 1 -2 3; 1 1 1 1; 0 1 2 3] \ exp.([ -1, -1, 1, 1])
    t = Polynomial(c)
    exact(x) = exp(x) - t(x)
    maxerr = maximum(abs(u(x) - exact(x)) for x in range( -1, 1, 801))
    ax.title = @sprintf("max err = %.5g", maxerr)
    return fig
end
```

Output 38

```
using SpectralMethodsTrefethen
p38()
```

0.14.2 Program 39

```
using CairoMakie, Printf
using LinearAlgebra, SpectralMethodsTrefethen, Polynomials
"p39 - eigenmodes of biharmonic on a square with clamped BCs (compare p38)"
function p39(N=17)
    # Construct spectral approximation to biharmonic operator:
    D, x = cheb(N)
    y = x
    Dš = (D^2)[2:N, 2:N]
    s = [1 / (1 - x^2) for x in x[2:N]]
    S = Diagonal([0; s; 0])
    D = (Diagonal(@. 1 - x ^ 2) * D^4 - Diagonal(8x) * D^3 - 12D^2) * S
    D = D[2:N, 2:N]
    \Deltaš = kron(I(N -1), D) + kron(D, I(N -1)) + 2kron(Dš, I(N -1)) * kron(I(N -
1), Dš)
```

```
    # Find and plot 25 eigenmodes:
    \lambda, W = eigen( \Deltaš)
    \lambda, W = real( \lambda[1:25]), real(W[:, 1:25])
    \lambda = sqrt.( \lambda / \lambda[1])
    xx = yy = -1:0.01:1
    square = [1 + 1im, -1 + 1im, -1 - 1im, 1 - 1im, 1 + 1im]
    fig = Figure(size=(640, 640))
    modes = reshape(1:25, 5, 5)'
    for (i, mode) in pairs(modes)
        str = @sprintf("%0.6g", \lambda[mode])
        V = zeros(N+1, N+1)
        V[2:N, 2:N] = reshape(W[:, mode], N -1, N -1)
        U = interp2dgrid(V, chebinterp, chebinterp, xx, yy)
        ax = Axis(fig[i[1], i[2]]; title=str, titlefont=:regular, aspect=DataAspect())
        contour!(ax, xx, yy, U; levels=[0], color=:darkblue, linewidth=1)
        lines!(ax, reim(square)...; linewidth=2, color=:black)
        hidedecorations!(ax)
        hidespines!(ax)
    end
    return fig
end
```

Output 39

```
p39()
```

0.14.3 Program 40

```
using CairoMakie, Printf, LaTeXStrings
using LinearAlgebra, SpectralMethodsTrefethen
"p40 - eigenvalues of OrrSommerfeld operator (compare p38)"
function p40(Re=5772)
    N = [40 60; 80 100]
    fig = Figure(size=(640, 640))
    ax = [Axis(fig[i, j]; aspect=1, limits=( -0.8, 0.2, -1, 0)) for i in 1:2, j in 1:2]
    for (i, N) in pairs(N)
        # 2nd - and 4th -order differentiation matrices:
        D, x = cheb(N)
        Dš = (D^2)[2:N, 2:N]
        S = Diagonal([0; (@. 1 / (1 - x[2:N]^2)); 0])
        D = (Diagonal(@. 1 - x^2) * D^4 - Diagonal(8x) * D^3 - 12D^2) * S
        D = D[2:N, 2:N]

        # OrrSommerfeld operators A,B and generalized eigenvalues:
        B = Dš - I
        A = (D - 2Dš + I) / Re - 2im * I - 1im * Diagonal(@. 1 - x[2:N]^2) * B
        \lambda = eigvals(A, B)
        vlines!(ax[i], [0], color=:gray)
        scatter!(ax[i], reim( \lambda)...; color=real( \lambda), colormap=:Blues, colorrange=(
0.8, 0) )
        str = @sprintf("\$N = %d,\\; \\lambda_\\mathrm{max}\$ = %.11f", N, maximum(real, \lamb
        ax[i].title = latexstring(str)
    end
    return fig
```